

快手

Q1. Nginx 配置

基本作用：

Nginx 是一款高性能的 Web 服务器，通常用于静态资源的托管、反向代理、负载均衡等。它的配置涉及到以下几个方面：

- **静态资源服务**：配置 Nginx 作为静态文件服务器，处理前端资源（HTML、CSS、JavaScript、图片等）的请求。
- **反向代理**：将客户端请求转发到后台的应用服务器（如 Node.js、Java、Spring Boot 等）。
- **负载均衡**：将请求分发到多个服务器，提高系统的性能和可用性。

基本配置示例：

```
server {  
    listen 80;  
    server_name example.com;  
  
    # 指定前端静态文件目录  
    location / {  
        root /usr/share/nginx/html;  
        index index.html;  
    }  
  
    # 反向代理到后端 API 服务器  
    location /api/ {  
        proxy_pass http://localhost:3000;  
        proxy_set_header Host $host;  
        proxy_set_header X-Real-IP $remote_addr;  
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;  
    }  
}
```

Q2. 页面路由跳转

- **前端路由模式**：
 - **Hash 模式**：通过 # 实现路由，如 `https://example.com/#/home`。无需服务器支持，兼容性好。
 - **History 模式**：使用 HTML5 History API（如 `pushState`），URL 更简洁（`https://example.com/home`）。需服务器配合，避免刷新时 404。
- **服务器配置**（以 Nginx 为例）：

```
location / {  
    try_files $uri $uri/ /index.html; # 所有未找到的路径返回 index.html  
}
```

- **框架实现**：

- Vue Router: `createWebHistory` (History 模式) 或 `createWebHashHistory`。
- React Router: `<BrowserRouter>` (需服务器配置) 或 `<HashRouter>`。

Q3. HTTPS 攻击方式

- 常见攻击手段:
 - 中间人攻击 (MITM) : 攻击者伪造证书或利用用户忽略证书错误, 劫持加密连接。防御: 强制校验证书有效性, 启用 HSTS。
 - 协议漏洞: 如心脏出血 (Heartbleed)、POODLE (SSLv3 漏洞)。防御: 禁用不安全协议 (如 SSLv3), 使用 TLS 1.2+。
 - 降级攻击: 迫使客户端使用弱加密算法。防御: 配置强加密套件 (如 AES-GCM), 禁用弱算法。
 - 证书伪造: 攻击者通过非法 CA 签发假证书。防御: 使用可信 CA, 定期更新证书。
- 关键配置: 严格选择加密套件、启用 HSTS (HTTP Strict Transport Security)、定期更新证书。

Q4. HTTP 缓存机制

- 强缓存: 直接使用本地副本, 不发送请求。
 - 响应头:
 - `Cache-Control: max-age=3600` (资源有效期为 3600 秒), 相对时间。
 - `Expires: wed, 21 Oct 2025 07:28:00 GMT` (过期时间, 优先级低于 `Cache-Control`), 绝对时间。
 - 应用场景: 静态资源 (如 JS/CSS 文件) 通过文件名哈希实现长期缓存 (`app.a1b2c3.js`)。
- 协商缓存: 向服务器验证资源是否变更, 返回 304 或新内容。
 - 响应头:
 - `Last-Modified` (资源最后修改时间) + 请求头 `If-Modified-Since`。
 - `ETag` (资源唯一标识) + 请求头 `If-None-Match` (优先级更高)。
- 缓存策略:
 - 频繁变动的资源: `Cache-Control: no-cache` (启用协商缓存)。
 - 长期不变的资源: `Cache-Control: public, max-age=31536000` (强缓存一年)。

Q5. 两栏布局并实现垂直居中的方法

方法 1: Flexbox (推荐)

使用 `display: flex` 和 `align-items: center` 让子元素垂直居中。

```
<!DOCTYPE html>
<html lang="zh">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Flexbox 两栏布局</title>
</head>
<body>
  <div class="container">
    <div class="box1">左侧内容</div>
    <div class="box2">右侧内容</div>
  </div>
</body>
</html>
```

```
.container {
  display: flex;
  align-items: center; /* 垂直居中 */
  justify-content: space-between; /* 左右排列 */
  height: 100vh; /* 让容器铺满整个视口高度 */
}
```

```

        padding: 20px;
    }
    .left {
        width: 30%;
        background-color: lightblue;
        padding: 20px;
    }
    .right {
        width: 65%;
        background-color: lightcoral;
        padding: 20px;
    }
}
</style>
</head>
<body>
    <div class="container">
        <div class="left">左侧内容</div>
        <div class="right">右侧内容</div>
    </div>
</body>
</html>

```

☑ 优势：

- 代码简洁，现代浏览器支持良好。
- `align-items: center` 实现垂直居中。
- `justify-content: space-between` 控制左右分布。

方法 2: Grid 布局

使用 `display: grid` 也可以很方便地实现两栏布局和垂直居中。

```

<style>
    .container {
        display: grid;
        grid-template-columns: 30% 65%;
        align-items: center; /* 垂直居中 */
        height: 100vh;
        padding: 20px;
    }
    .left {
        background-color: lightblue;
        padding: 20px;
    }
    .right {
        background-color: lightcoral;
        padding: 20px;
    }
</style>
<div class="container">
    <div class="left">左侧内容</div>
    <div class="right">右侧内容</div>
</div>

```

☑ 优势：

- `grid-template-columns` 控制两栏宽度。
- `align-items: center` 实现垂直居中。
- 适用于更复杂的布局。

方法 3: 绝对定位 + transform

使用 `position: absolute` 和 `transform: translateY(-50%)` 进行垂直居中。

```
<style>
  .container {
    position: relative;
    height: 100vh;
    padding: 20px;
  }
  .left, .right {
    position: absolute;
    top: 50%;
    transform: translateY(-50%);
    padding: 20px;
  }
  .left {
    left: 0;
    width: 30%;
    background-color: lightblue;
  }
  .right {
    right: 0;
    width: 65%;
    background-color: lightcoral;
  }
</style>
<div class="container">
  <div class="left">左侧内容</div>
  <div class="right">右侧内容</div>
</div>
```

✓ 适用场景:

- 适用于固定宽度的左右栏。
- `top: 50% + transform: translateY(-50%)` 实现垂直居中。

方法 4: 使用 table

可以利用 `display: table` 和 `vertical-align: middle` 进行居中。

```
<style>
  .container {
    display: table;
    width: 100%;
    height: 100vh;
  }
  .row {
    display: table-row;
  }
</style>
```

```
.left, .right {
  display: table-cell;
  vertical-align: middle; /* 垂直居中 */
  padding: 20px;
}
.left {
  width: 30%;
  background-color: lightblue;
}
.right {
  width: 65%;
  background-color: lightcoral;
}
</style>
<div class="container">
  <div class="row">
    <div class="left">左侧内容</div>
    <div class="right">右侧内容</div>
  </div>
</div>
```

✔ 适用场景：

- 兼容老旧浏览器（IE8+）。
- 不推荐用于现代布局（推荐 Flexbox）。

总结

方法	适用场景	兼容性	代码简洁度
Flexbox ✔	现代布局推荐，适用于大多数场景	IE10+	简洁
Grid ✔	复杂布局，适用于栅格系统	IE11+	简洁
绝对定位 + transform	固定宽高的两栏布局	IE9+	适中
table	兼容老旧浏览器（不推荐）	IE8+	复杂

如果是现代布局，推荐使用 Flexbox 或 Grid。如果需要支持老旧浏览器，可以考虑 position: absolute 或 display: table。

Q6. 异步函数有哪些

1. 回调函数（Callback）

- 原理：将函数作为参数传递给异步任务，任务完成后调用该函数。
- 场景：传统异步操作（如 setTimeout、Node.js 文件读写）。
- 示例：

```
setTimeout(() => {
  console.log('延迟1秒执行');
}, 1000);

// Node.js 文件读取
fs.readFile('file.txt', 'utf8', (err, data) => {
  if (err) throw err;
  console.log(data);
});
```

- **缺点：**回调地狱（Callback Hell），难以维护和调试。

2. Promise

- **原理：**用对象表示异步操作的最终完成或失败，通过链式调用（`.then()`、`.catch()`）管理异步流程。
- **场景：**替代回调嵌套，处理顺序或并行的异步任务。
- **示例：**

```
fetch('https://api.example.com/data')
  .then(response => response.json())
  .then(data => console.log(data))
  .catch(error => console.error('请求失败:', error));

// 并行处理多个 Promise
Promise.all([promise1, promise2])
  .then(results => console.log(results));
```

- **优点：**链式调用、集中错误处理（`.catch()`）。
- **缺点：**无法取消，需手动处理竞态条件。

3. Async/Await

- **原理：**基于 Promise 的语法糖，用同步写法处理异步操作，`async` 函数返回 Promise，`await` 等待 Promise 完成。
- **场景：**简化异步代码逻辑，处理复杂流程。
- **示例：**

```
async function fetchData() {
  try {
    const response = await fetch('https://api.example.com/data');
    const data = await response.json();
    console.log(data);
  } catch (error) {
    console.error('请求失败:', error);
  }
}
```

- **优点：**代码简洁，错误处理直观（`try/catch`）。
- **注意：**`await` 必须在 `async` 函数内使用。

4. Generator 函数

- **原理**: 通过 `function*` 定义生成器函数, 使用 `yield` 暂停执行, 配合 `next()` 逐步恢复。
- **场景**: 复杂流程控制 (如按需加载数据), 是 Async/Await 的前身。
- **示例**:

```
function* fetchGenerator() {  
  const data1 = yield fetch('/api/data1');  
  const data2 = yield fetch('/api/data2');  
  console.log(data1, data2);  
}  
  
// 手动执行 Generator  
const gen = fetchGenerator();  
gen.next().value  
  .then(res => res.json())  
  .then(data1 => gen.next(data1).value)  
  .then(res => res.json())  
  .then(data2 => gen.next(data2));
```

- **缺点**: 需手动控制执行流程, 代码复杂度高。

5. 事件监听 (Event Emitter)

- **原理**: 通过事件驱动模型, 监听特定事件并触发回调。
- **场景**: DOM 事件、Node.js 的 `EventEmitter` 模块。
- **示例**:

```
// 浏览器 DOM 事件  
button.addEventListener('click', () => {  
  console.log('按钮被点击');  
});  
  
// Node.js EventEmitter  
const EventEmitter = require('events');  
const emitter = new EventEmitter();  
emitter.on('data', data => console.log(data));  
emitter.emit('data', 'Hello World');
```

- **适用性**: 适合松散耦合的场景 (如用户交互、消息通知)。

6. 发布订阅 (Pub/Sub) 模式

- **原理**: 解耦发布者和订阅者, 通过中间层管理事件。
- **场景**: 跨组件通信、全局状态管理 (如 Vue 的 EventBus、Redux)。
- **示例**:

```
// 简易实现  
const pubSub = {
```

```
events: {},
subscribe(event, callback) {
  this.events[event] = this.events[event] || [];
  this.events[event].push(callback);
},
publish(event, data) {
  (this.events[event] || []).forEach(callback => callback(data));
}
};

pubSub.subscribe('message', data => console.log(data));
pubSub.publish('message', 'Hello!');
```

7. Web Workers

- **原理：**浏览器多线程机制，在后台线程执行脚本，通过 `postMessage` 通信。
- **场景：**CPU 密集型任务（如大数据处理），避免阻塞主线程。
- **示例：**

```
// 主线程
const worker = new Worker('worker.js');
worker.postMessage({ data: 'start' });
worker.onmessage = e => console.log(e.data);

// worker.js
self.onmessage = e => {
  const result = heavyCalculation(e.data);
  self.postMessage(result);
};
```

8. RxJS (Observable)

- **原理：**基于观察者模式和迭代器模式，提供响应式编程模型。
- **场景：**复杂异步流处理（如事件节流、取消、组合）。
- **示例：**

```
import { fromEvent, interval } from 'rxjs';
import { debounceTime, mergeMap } from 'rxjs/operators';

fromEvent(input, 'input')
  .pipe(
    debounceTime(300),
    mergeMap(e => fetch(`/api/search?q=${e.target.value}`))
  )
  .subscribe(res => console.log(res));
```

- **优点：**强大的异步流控制能力，适合高频事件（如搜索框输入）。
-

总结回答技巧

1. **分层回答**：从基础到高级（回调 → Promise → Async/Await → 高级模式）。
2. **结合场景**：说明每种方法的适用场景（如“Promise 适合链式请求，Async/Await 简化代码”）。
3. **对比优缺点**：例如“回调简单但易嵌套，Promise 解决了回调地狱但需手动处理竞态”。
4. **实际案例**：用简短代码片段展示用法（如用 Async/Await 封装网络请求）。
5. **扩展知识**：提到底层原理（如事件循环、微任务队列）会加分。

Q7. call、apply 和 bind

call、apply 和 bind 都是用于显式绑定函数执行时的 this 值的方法。

1. call 方法

- **功能**：立即调用函数，显式指定 this 值，并逐个传递参数。
- **语法**：

```
func.call(thisArg, arg1, arg2, ...)
```

- **示例**：

```
function introduce(greeting, role) {  
  console.log(`${greeting}, 我是${this.name}, 职业是${role}。`);  
}  
  
const person = { name: "张三" };  
introduce.call(person, "你好", "工程师");  
// 输出：你好，我是张三，职业是工程师。
```

2. apply 方法

- **功能**：立即调用函数，显式指定 this 值，参数以数组形式传递。
- **语法**：

```
func.apply(thisArg, [argsArray])
```

- **示例**：

```
function sum(a, b, c) {  
  return a + b + c;  
}  
  
const numbers = [1, 2, 3];  
console.log(sum.apply(null, numbers)); // 输出：6  
  
// ES6后可用展开运算符替代：  
console.log(sum(...numbers)); // 输出：6
```

3. bind 方法

- 功能：返回一个新函数，永久绑定 `this` 值和预设参数，不立即执行。
- 语法：

```
const boundFunc = func.bind(thisArg, arg1, arg2, ...)
```

- 示例：

```
const person = { name: "李四" };

function showName() {
  console.log(this.name);
}

const boundShow = showName.bind(person);
boundShow(); // 输出：李四

// 预设参数（柯里化）：
function multiply(a, b) {
  return a * b;
}
const double = multiply.bind(null, 2);
console.log(double(3)); // 输出：6
```

核心区别对比

特性	call	apply	bind
执行时机	立即执行	立即执行	返回函数，延迟执行
参数形式	参数列表（逐个传递）	参数数组	参数列表或预设
返回结果	函数返回值	函数返回值	绑定后的新函数
典型场景	明确参数数量时	参数动态或已存在于数组时	需要保留 this 或预设参数

实际应用场景

1. call 的典型应用

- 借用其他对象的方法：

```
const obj1 = { data: "数据" };
const obj2 = { getData: function() { return this.data; } };

console.log(obj2.getData.call(obj1)); // 输出："数据"
```

2. apply 的典型应用

- 数组合并或数学计算：

```
const arr1 = [1, 2, 3];
const arr2 = [4, 5, 6];
arr1.push.apply(arr1, arr2); // ES6前合并数组
console.log(arr1); // 输出: [1, 2, 3, 4, 5, 6]
```

3. bind 的典型应用

- 事件处理或回调函数绑定 this：

```
class Button {
  constructor() {
    this.label = "点击我";
    this.element = document.createElement("button");
    this.element.addEventListener("click", this.handleClick.bind(this));
  }

  handleClick() {
    console.log(this.label); // 正确指向Button实例
  }
}
```

注意事项

1. **箭头函数**：无法通过 call、apply、bind 改变 this，其 this 由词法作用域决定。

```
const obj = { value: 10 };
const arrowFunc = () => console.log(this.value);
arrowFunc.call(obj); // 输出: undefined（严格模式）或全局对象的值
```

2. **性能优化**：频繁使用 bind 可能生成多个函数对象，需注意内存开销。

3. **参数处理**：

- call 和 apply 的第一个参数为 null 或 undefined 时，非严格模式下 this 指向全局对象。
- bind 后的函数仍可接收剩余参数：

```
const greet = function(greeting, name) {
  console.log(greeting + ", " + name);
};
const sayHello = greet.bind(null, "Hello");
sayHello("Alice"); // 输出: Hello, Alice
```

Q8. 介绍一下flex

Flex 弹性布局通过容器和项目的属性组合，快速实现元素的排列对齐。比如用 justify-content: center 和 align-items: center 轻松居中元素，用 flex:1 实现等分空间。相比浮动布局，它避免了清除浮动和兼容性问题，代码更简洁。我在项目中常用它实现导航栏、卡片列表和响应式布局。

1. 一句话定义

"Flex 弹性布局是 CSS3 提供了一种一维布局模型，通过简单的属性设置即可快速实现复杂排列、对齐和空间分配，解决了传统布局（如浮动、定位）的代码冗余和兼容性问题。"

2. 核心概念 (30秒)

- 两根轴：
 - 主轴 (Main Axis)：由 `flex-direction` 定义方向（默认水平，可切换为垂直）。
 - 交叉轴 (Cross Axis)：始终垂直于主轴，控制垂直对齐。
- 容器与项目：
 - 容器 (Container)：设置 `display: flex` 的父元素，控制全局排列规则。
 - 项目 (Item)：直接子元素，自动成为弹性项，可单独调整特性（如伸缩比例）。

3. 核心属性 (30秒)

- 容器级控制：
 - `justify-content`：主轴对齐（如居中、两端对齐）。
 - `align-items`：交叉轴对齐（如垂直居中）。
 - `flex-wrap`：换行控制（多行布局）。
- 项目级控制：
 - `flex: 1`：等分剩余空间（经典等宽布局）。
 - `order`：调整项目显示顺序。
 - `align-self`：覆盖容器对齐规则。

4. 典型应用场景 (20秒)

- 导航栏：结合 `justify-content: space-between` 实现左右分离。
- 垂直居中：`align-items: center` + `justify-content: center`。
- 响应式列表：`flex-wrap: wrap` + `flex-basis` 控制自适应换行。
- 圣杯布局：容器设为 `flex-direction: column`，主内容区用 `flex: 1` 撑满。

5. 对比与总结 (10秒)

- 优势：代码简洁、自适应强、完美解决传统布局痛点。
- 局限：一维布局场景（二维布局优先选 Grid）。
- 一句话收尾："Flex 是我日常开发的首选布局方案，尤其在移动端和组件化场景中，能极大提升开发效率。"

前端开发中的设计模式有助于提升代码的可读性、可维护性和复用性。以下是一些常见的前端设计模式：

Q9：设计模式

1. 模块化模式 (Module Pattern)

用于封装私有变量和方法，避免全局污染。

示例 (IIFE 立即执行函数)：

```
const Module = (function () {  
    let privateVar = "我是私有变量";  
  
    function privateMethod() {  
        console.log("我是私有方法");  
    }  
  
    return {  
        publicMethod: function () {  
            console.log("我是公有方法，可以访问：", privateVar);  
        }  
    };  
})();  
  
Module.publicMethod();
```

适用场景：

- 需要私有作用域的情况下，比如避免全局变量污染。

2. 单例模式 (Singleton Pattern)

确保一个类只有一个实例，并提供全局访问点。

示例：

```
const Singleton = (function () {  
    let instance;  
  
    function createInstance() {  
        return { name: "我是唯一的实例" };  
    }  
  
    return {  
        getInstance: function () {  
            if (!instance) {  
                instance = createInstance();  
            }  
            return instance;  
        }  
    };  
})();  
  
const instance1 = Singleton.getInstance();  
const instance2 = Singleton.getInstance();  
console.log(instance1 === instance2); // true
```

适用场景：

- Vuex、Redux 中的 Store。

- 需要共享数据的全局对象（如缓存、配置管理）。

3. 观察者模式 (Observer Pattern)

一种订阅-发布模式，一个对象 (Subject) 维护一组依赖它的对象 (Observers)，当状态改变时通知所有观察者。

示例：

```
class Subject {
  constructor() {
    this.observers = [];
  }

  addObserver(observer) {
    this.observers.push(observer);
  }

  notify(data) {
    this.observers.forEach(observer => observer.update(data));
  }
}

class Observer {
  update(data) {
    console.log("接收到通知:", data);
  }
}

// 使用示例
const subject = new Subject();
const observer1 = new Observer();
const observer2 = new Observer();

subject.addObserver(observer1);
subject.addObserver(observer2);

subject.notify("数据更新了!");
```

适用场景：

- Vue 的响应式数据绑定（如 `watch`）。
- 发布-订阅模式（如 `EventEmitter`、`Event Bus`）。

4. 工厂模式 (Factory Pattern)

通过工厂方法创建对象，而不是直接使用 `new` 关键字。

示例：

```
class User {
  constructor(name, role) {
    this.name = name;
    this.role = role;
  }
}
```

```

    describe() {
        console.log(`${this.name} 是 ${this.role}`);
    }
}

class UserFactory {
    static createUser(name, role) {
        return new User(name, role);
    }
}

// 使用工厂创建对象
const user1 = UserFactory.createUser("小明", "前端工程师");
const user2 = UserFactory.createUser("小红", "后端工程师");

user1.describe();
user2.describe();

```

适用场景：

- 组件创建（如 React/Vue 组件工厂）。
- 动态创建不同类型的对象（如不同用户角色、图表类型）。

5. 代理模式 (Proxy Pattern)

代理对象控制对目标对象的访问，可以增加权限控制、缓存等功能。

示例（数据缓存代理）：

```

class DataFetcher {
    fetchData(url) {
        console.log(`从 ${url} 获取数据...`);
        return `数据来自 ${url}`;
    }
}

class DataProxy {
    constructor() {
        this.cache = {};
        this.fetcher = new DataFetcher();
    }

    fetchData(url) {
        if (this.cache[url]) {
            console.log("从缓存返回数据...");
            return this.cache[url];
        }
        this.cache[url] = this.fetcher.fetchData(url);
        return this.cache[url];
    }
}

// 使用代理
const proxy = new DataProxy();
console.log(proxy.fetchData("https://api.example.com/data"));

```

```
console.log(proxy.fetchData("https://api.example.com/data")); // 缓存返回
```

适用场景：

- Vue/React 组件的 `computed` 计算属性缓存。
- API 请求缓存，避免重复请求。

6. 适配器模式 (Adapter Pattern)

将不同接口格式的数据转换为统一格式，使得不兼容的接口可以协同工作。

示例：

```
class OldAPI {
  getData() {
    return { name: "小明", age: 20 };
  }
}

class Adapter {
  constructor(oldApi) {
    this.oldApi = oldApi;
  }

  getUser() {
    const data = this.oldApi.getData();
    return {
      username: data.name,
      userAge: data.age
    };
  }
}

// 使用适配器
const oldApi = new OldAPI();
const adapter = new Adapter(oldApi);
console.log(adapter.getUser()); // { username: '小明', userAge: 20 }
```

适用场景：

- 兼容旧版 API 数据格式转换。
- 统一不同数据结构的处理方式。

7. 装饰器模式 (Decorator Pattern)

在不修改原对象的情况下，动态添加功能。

示例（对函数进行装饰）：

```
function logDecorator(fn) {
  return function (...args) {
    console.log("调用前：参数", args);
    const result = fn(...args);
    console.log("调用后：结果", result);
    return result;
  };
}
```

```
};  
}  
  
function add(a, b) {  
  return a + b;  
}  
  
const decoratedAdd = logDecorator(add);  
decoratedAdd(2, 3);
```

适用场景：

- Vue/React 中的高阶组件（HOC）。
- ES7 装饰器（如 `@log`）。

8. MVC/MVVM 模式

用于前端框架架构设计。

MVC (Model-View-Controller)

- **Model (数据层)**：存储和管理数据（如 `state`）。
- **View (视图层)**：用户界面（如 HTML/CSS/JS）。
- **Controller (控制器)**：处理用户交互，更新 `Model` 并通知 `View`。

适用场景：早期前端架构（如 jQuery + 后端渲染）。

MVVM (Model-View-ViewModel)

- **Model**：数据模型（如 Vue/React 组件的 `state`）。
- **View**：用户界面（如 Vue/React 组件模板）。
- **ViewModel**：双向数据绑定，连接 `Model` 和 `View`（如 Vue 的 `computed` 和 `watch`）。

适用场景：Vue、React（使用 Hooks/State 管理）。

9. 总结

设计模式	适用场景
模块化模式	避免全局变量污染
单例模式	Vuex/Redux，应用配置对象
观察者模式	发布-订阅、事件总线
工厂模式	组件/对象的创建
代理模式	数据缓存、权限控制
适配器模式	兼容不同 API
装饰器模式	React HOC、函数增强
MVC/MVVM	Vue、React 应用架构

Q10: Vue 和 React的区别

1. 框架 vs. 库

- Vue 是一个**渐进式框架**，提供完整的解决方案，包括路由（Vue Router）和状态管理（Vuex/Pinia）。
- React 是一个**前端库**，主要用于构建 UI 组件，需要配合其他库（如 React Router、Redux）才能实现完整功能。

2. 语法风格

- Vue 采用 **模板语法（基于 HTML）**，如：`<template> + {{ 数据 }} + v-if/v-for` 等指令，更符合传统前端开发习惯。
- React 主要使用 **JSX 语法**，即 HTML 写在 JavaScript 中，如：`{变量} + map() + 条件渲染（三元运算符）`，更加灵活但学习成本稍高。

3. 数据绑定

- Vue 采用 **双向数据绑定（v-model）**，适合表单和数据同步场景。
- React 主要是 **单向数据流**，需要手动更新 `state` 以同步 UI 变化。

4. 状态管理

- Vue 内置 `ref`、`reactive`，配合 Vuex/Pinia 进行全局状态管理。
- React 主要使用 `useState`、`useReducer`，或 Redux、Recoil 进行状态管理。

5. 组件通信

- Vue 使用 `props` 传递数据，`$emit` 触发事件，也支持 `provide/inject`。
- React 主要用 `props` 传递，配合 `context API` 或 Redux 进行跨组件通信。

6. 生态和应用场景

- Vue 在国内更受欢迎，适合中小型项目，开发成本较低。
- React 在国外广泛应用，大型项目（如 Facebook、Netflix）较多，社区生态更丰富。

总结来说，Vue 更易上手，适合快速开发；React 更灵活，适用于复杂项目。

Q11: WebSocket 心跳机制的实现与作用

1. 作用

WebSocket 连接是**长连接**，但可能由于**网络异常**、**服务器/客户端崩溃**、**NAT 设备超时**等原因导致连接断开，而应用层无法感知。因此，**心跳机制**的作用是：

- 保持连接活跃**，防止 NAT 设备超时断开连接。
- 检测连接状态**，及时发现并重连 WebSocket。
- 避免无效连接**，减少服务器资源占用。

2. 实现方式

心跳机制一般有**客户端主动发送**和**服务端被动响应**两种方式：

方法 1: Ping-Pong 机制（推荐）

WebSocket 协议**内置**了 `ping` 和 `pong` 帧，可用于心跳检测。

- 客户端发送** `ping`，服务器收到后**自动回复** `pong`。
- 如果客户端**一段时间没有收到** `pong`，说明连接可能断开了，需要重连。

示例（前端 JavaScript 实现）：

```
const ws = new WebSocket("wss://example.com/socket");
```

```

ws.onopen = function () {
    console.log("WebSocket 连接成功");
    startHeartbeat();
};

ws.onmessage = function (event) {
    console.log("收到消息:", event.data);
};

ws.onclose = function () {
    console.log("WebSocket 连接关闭, 尝试重连...");
    reconnect();
};

ws.onerror = function (error) {
    console.log("WebSocket 发生错误:", error);
    ws.close();
};

// 发送心跳
function startHeartbeat() {
    setInterval(() => {
        if (ws.readyState === WebSocket.OPEN) {
            ws.send("ping"); // 发送心跳
            console.log("发送心跳包");
        }
    }, 30000); // 每 30 秒发送一次
}

// 断线重连
function reconnect() {
    setTimeout(() => {
        console.log("正在尝试重连...");
        ws = new WebSocket("wss://example.com/socket");
    }, 5000); // 5 秒后重连
}

```

示例 (Node.js 服务端实现, `ws` 库) :

```

const WebSocket = require('ws');
const wss = new WebSocket.Server({ port: 8080 });

wss.on('connection', function (ws) {
    console.log("客户端连接成功");

    ws.on('message', function (message) {
        if (message === "ping") {
            ws.send("pong"); // 服务器响应心跳
        } else {
            console.log("收到消息:", message);
        }
    });

    ws.on('close', function () {
        console.log("客户端断开连接");
    });
});

```

```
});
```

方法 2：定时发送自定义心跳包

如果服务器不支持 `ping / pong`，可以自己定义心跳消息：

- **客户端** 每隔一段时间发送 `{ type: "heartbeat" }`
- **服务器** 收到后返回 `{ type: "heartbeat_ack" }`
- 如果客户端没有收到响应，则尝试重连。

示例（前端 JavaScript 实现）：

```
function startHeartbeat() {
  setInterval(() => {
    if (ws.readyState === WebSocket.OPEN) {
      ws.send(JSON.stringify({ type: "heartbeat" })); // 发送心跳
      console.log("发送心跳包");
    }
  }, 30000); // 30 秒发送一次
}

ws.onmessage = function (event) {
  let data = JSON.parse(event.data);
  if (data.type === "heartbeat_ack") {
    console.log("收到服务器心跳响应");
  } else {
    console.log("收到消息:", data);
  }
};
```

3. 总结

方式	说明	优点	缺点
<code>ping-pong</code>	WebSocket 内置的机制	轻量、高效	部分 WebSocket 库可能不支持
自定义心跳包	发送 JSON 格式心跳消息	灵活可定制	需要手动解析消息

- 推荐 `ping-pong` 机制（WebSocket 内置支持，简单可靠）。
- 如果服务器不支持 `ping-pong`，可以使用自定义心跳包。
- 客户端定期检测心跳，若断开则重连，保证 WebSocket 连接稳定。

Q12：CDN 与 Service Worker

1. CDN（内容分发网络）

概念

CDN（Content Delivery Network）是一种 **分布式网络**，通过在全球范围内部署 **边缘服务器** 来加速内容传输，减少延迟。

作用

- **加速访问**：就近访问 CDN 节点，提高页面加载速度。
- **降低服务器压力**：减少源站负载，提升可用性。
- **提高稳定性**：即使源站宕机，CDN 也能继续提供缓存内容。
- **节省带宽**：减少重复请求，降低带宽消耗。

CDN 如何应对缓存更新？

CDN 采用 **缓存机制** 来减少请求，但当资源更新时，需要处理缓存失效问题，常见方法有：

1. URL 版本号（推荐）

- 更新时，在 URL 末尾添加 hash 或时间戳，例如：

```
<script src="app.js?v=20250317"></script>
```

- 每次构建更新时，文件名变化，避免 CDN 继续使用旧缓存。

2. CDN 过期策略（Cache-Control + ETag）

- **Cache-Control: max-age=3600, must-revalidate**：控制缓存过期时间。
- **ETag**：服务器计算文件哈希值，变更时通知浏览器重新获取。

3. 主动刷新缓存（CDN 提供的 API）

- 部分 CDN（如 Cloudflare、阿里云 CDN）提供 **API 手动清除缓存**，但成本较高，通常用于紧急情况。

2. Service Worker

概念

Service Worker 是 **浏览器运行在后台的独立脚本**，可以拦截和控制网络请求，实现**离线缓存**、**消息推送**、**后台同步**等功能。

作用

- **缓存拦截**：拦截 HTTP 请求，提供离线访问能力（如 PWA 应用）。
- **性能优化**：减少网络请求，提高页面加载速度。
- **消息推送（Push Notification）**：即使页面关闭，也能收到服务器推送的通知。
- **后台数据同步（Background Sync）**：用户离线时存储操作，在线后自动同步（如聊天应用）。

Service Worker 除了做请求拦截缓存，还有哪些用途？

1. 消息推送（Push API）

- 可以通过 **Web Push API** 监听服务器推送的通知，即使页面关闭也能收到。
- 适用于 **新闻通知**、**社交消息提醒** 等场景。
- 示例：

```
self.addEventListener("push", function (event) {  
  const options = { body: "你有一条新消息！" };  
  event.waitUntil(self.registration.showNotification("提醒", options));  
});
```

2. 后台数据同步（Background Sync API）

- 允许在用户离线时存储请求，等网络恢复后自动执行。
- 适用于 **离线操作（如提交表单、聊天消息存储）**。
- 示例：

```
self.addEventListener('sync', function(event) {
  if (event.tag === 'sync-messages') {
    event.waitUntil(sendPendingMessages());
  }
});
```

3. 拦截和修改请求 (Request/Response Manipulation)

- Service Worker 可以拦截请求，**修改 headers、返回 mock 数据**，提升安全性或调试应用。
- 示例（拦截 API 请求并返回自定义数据）：

```
self.addEventListener("fetch", (event) => {
  if (event.request.url.includes("/api/data")) {
    event.respondWith(new Response(JSON.stringify({ message: "Hello from SW" })));
  }
});
```

4. PWA (渐进式 Web 应用) 支持

- 结合 **App Manifest**，让网站支持**离线访问**，并可以安装到桌面，如 Twitter Lite、Google Maps PWA。

3. 面试回答示例

你能介绍一下 CDN 和 Service Worker 吗？

- **CDN 是 内容分发网络**，通过全球服务器加速资源加载，减少源站压力，提高可用性。
- **Service Worker** 是浏览器后台运行的独立脚本，常用于**拦截请求、缓存资源、离线访问**等。

CDN 是如何解决缓存更新问题的？

- **最佳实践是 URL 版本号（如 `app.js?v=123`）**，确保每次更新文件名不同。
- 还可以使用 **Cache-Control** 和 **ETag** 控制缓存刷新策略。
- 部分 CDN（如 Cloudflare、阿里云 CDN）支持**主动清除缓存**，但成本较高。

Service Worker 除了做请求拦截缓存，还有什么作用？

- **消息推送（Push API）**，即使页面关闭，也能接收服务器推送的通知。
- **后台数据同步（Background Sync API）**，离线时存储请求，网络恢复后自动执行。
- **请求拦截和修改**，可以**拦截 API 调用**，返回 mock 数据或优化请求。
- **支持 PWA**，让 Web 应用可以**离线运行并安装到桌面**，提升用户体验。

在面试中，回答 `code split`（代码拆分）和 `chunk`（代码块）相关问题时，建议从 **概念、作用、常见配置、应用场景** 以及 **优化方案** 等方面展开，既体现基础知识，又能展现优化思维。

Q13: Code Splitting

1. 什么是 Code Splitting（代码拆分）？

Code Splitting（代码拆分）是 Webpack 等构建工具提供的一种优化技术，将 JavaScript 代码拆分成多个 **小块（chunk）**，按需加载，提高页面性能。

作用

1. **减少首屏加载时间**：避免一次性加载所有代码，只加载当前页面或功能所需的部分。
2. **提高缓存效率**：拆分后的代码块可独立缓存，未变更的代码可复用，减少重复下载。
3. **按需加载（Lazy Loading）**：只有在用户访问某个功能时才加载相关代码，优化资源使用。

2. Code Splitting 常见方式

Webpack 提供三种方式进行代码拆分：

1) 入口分离（Entry Points）

手动定义多个 `entry` 入口，将代码拆分到不同的 `bundle.js`，适用于**多页应用（MPA）**。

```
module.exports = {
  entry: {
    home: "./src/home.js",
    about: "./src/about.js",
  },
  output: {
    filename: "[name].bundle.js",
  },
};
```

- 结果：生成 `home.bundle.js` 和 `about.bundle.js`，各页面按需加载对应的 JS 代码。

2) 动态导入（Dynamic Import）

适用于 **单页应用（SPA）**，按需加载模块，常用于路由懒加载：

```
import("./moduleA").then((module) => {
  module.default();
});
```

3) 代码共享（SplitChunksPlugin）

Webpack 内置 `SplitChunksPlugin`，可以拆分 **共享依赖**，减少重复打包：

```
module.exports = {
  optimization: {
    splitChunks: {
      chunks: "all", // 拆分同步和异步代码
      minSize: 20000, // 最小拆分大小（20KB）
      maxSize: 250000, // 最大拆分大小（250KB）
      minChunks: 1, // 至少被引用 1 次就拆分
      cacheGroups: {
        vendors: {
          test: /[\\/]node_modules[\\/]/, // 拆分第三方库
          name: "vendors",
        },
      },
    },
  },
};
```

```
    priority: -10, // 优先级高
  },
  commons: {
    test: /[\\/]src[\\/]$/,
    name: "commons",
    minChunks: 2, // 公共代码至少引用 2 次才拆分
    priority: -20,
  },
},
},
},
};
```

- 结果：
 - `node_modules` 里的代码会被拆分到 `vendors.js`。
 - 多个模块共用的代码 会被拆分到 `commons.js`，避免重复加载。

3. Code Splitting 适用场景

场景	拆分方式	作用
多页面应用 (MPA)	入口分离	为不同页面生成独立 JS，减少加载无关代码
单页面应用 (SPA)	路由懒加载	仅在访问某个页面时才加载相应代码，减少首屏体积
大型组件库	动态 <code>import()</code>	仅按需加载组件，提高页面交互体验
第三方库优化	<code>SplitChunksPlugin</code>	拆分 <code>node_modules</code> 代码，提高缓存复用
按用户权限加载	动态 <code>import()</code>	仅加载当前用户有权限访问的代码，减少不必要的 JS

4. 面试回答示例

面试官：Code Split 代码拆分的作用是什么？

Code Splitting 可以减少首屏加载时间，提高缓存复用率，并支持按需加载，从而优化前端性能。

面试官：Webpack 里如何进行代码拆分？

Webpack 提供了三种方式：

1. **入口分离：**适用于多页应用，每个页面生成独立的 `bundle.js`。
2. **动态导入 (`import()`)：**适用于 SPA 按需加载组件，结合 `React.lazy()` 实现路由懒加载。
3. **`SplitChunksPlugin`：**用于拆分第三方库和公共代码，提高缓存效率。

面试官：`SplitChunksPlugin` 里 `cacheGroups` 是如何工作的？

- `vendors` 规则用于拆分 `node_modules` 里的代码，优先级 `-10`，避免多次打包相同库。
- `commons` 规则用于拆分项目中被多个模块引用的代码，`minChunks: 2`，提高缓存利用率。

面试官：在 SPA 中，如何优化 Code Splitting 方案？

- 1. **首屏代码优化**: 使用 `React.lazy()` + `Suspense` 懒加载非核心组件。
- 2. **预加载关键模块**: 使用 `import(/* webpackPrefetch: true */ './Chart')` 提前加载核心代码。
- 3. **避免过度拆分**: 合理设置 `minSize` 和 `maxSize`, 防止 chunk 过小影响加载效率。

5. 总结

- **Code Splitting (代码拆分)** 主要用于**减少首屏加载时间、提高缓存效率、按需加载**, 提升前端性能。
- Webpack 提供三种拆分方式:
 - **入口分离**: MPA 适用, 每个页面独立 `bundle.js`。
 - **动态导入** (`import()`): SPA 适用, 路由懒加载、按需加载组件。
 - **SplitChunksPlugin**: 拆分第三方库和共享代码, 提高缓存利用率。
- 优化方案:
 - **webpackPrefetch / Preload** 提前加载关键模块。
 - **合理配置 SplitChunksPlugin**, 平衡 chunk 大小, 提升缓存利用率。

Q14: 浏览器存储存储方式、场景及 IndexedDB 数据结构更新

1. 浏览器存储方式及应用场景

常见存储方式对比

存储方式	容量	生命周期	作用范围	是否随 HTTP 传输	适用场景
Cookie	~4KB	可设置过期时间	同源共享	☑ 是, 默认随请求携带	会话管理、身份验证
LocalStorage	~5MB	永久存储	同源共享	✗ 否	用户偏好设置、静态数据缓存
SessionStorage	~5MB	仅当前会话	单个页面 (Tab)	✗ 否	多 Tab 数据隔离
IndexedDB	取决于磁盘空间	永久存储	同源共享	✗ 否	大数据存储, 离线缓存
Cache API	取决于磁盘空间	持久存储	同源共享	✗ 否	PWA、静态资源缓存

适用场景

- **Cookie**: 适用于 **身份验证 (如 JWT Token)**, 但体积小, 容易影响 HTTP 传输性能。
- **LocalStorage**: 适用于**存储小型静态数据**, 如用户偏好、主题设置等。
- **SessionStorage**: 适用于**存储当前 Tab 相关数据**, 如表单填写状态、临时用户数据等。
- **IndexedDB**: 适用于**存储结构化数据、大型 JSON、离线数据** (如 PWA 应用)。

- **Cache API**: 适用于**缓存静态资源** (如 HTML、CSS、JS、图片) , 加速页面加载, 提升 PWA 体验。

2. IndexedDB 介绍

特点

- **键值存储 (NoSQL 形式)**: 支持对象存储, 结构类似数据库, 支持事务处理。
- **支持索引**: 可以创建索引提高查询效率。
- **异步操作**: 使用 **事件 (onSuccess, onError) 或 Promise (indexedDB.open())** 进行数据操作。
- **可存储二进制数据**: 如 Blob、ArrayBuffer。

基本操作

```
const request = indexedDB.open("MyDatabase", 1);

request.onupgradeneeded = function (event) {
  const db = event.target.result;
  if (!db.objectStoreNames.contains("users")) {
    const store = db.createObjectStore("users", { keyPath: "id" });
    store.createIndex("nameIndex", "name", { unique: false });
  }
};

request.onsuccess = function (event) {
  const db = event.target.result;
  const transaction = db.transaction(["users"], "readwrite");
  const store = transaction.objectStore("users");
  store.add({ id: 1, name: "Alice" });
};
```

3. IndexedDB 如何应对数据结构更新

在 IndexedDB 中, **数据结构一旦定义, 不能直接修改** (例如不能直接添加字段、修改 keyPath) , 需要使用**数据库版本升级 (onupgradeneeded)** 来更新结构。

更新数据表结构

当需要增加新字段或索引时, 应当 **提高数据库版本号**:

```
const request = indexedDB.open("MyDatabase", 2); // 版本从 1 升级到 2

request.onupgradeneeded = function (event) {
  const db = event.target.result;
  const store = event.target.transaction.objectStore("users");

  // 添加新索引
  if (!store.indexNames.contains("ageIndex")) {
    store.createIndex("ageIndex", "age", { unique: false });
  }
};
```

◆ 原理:

- `onupgradeneeded` 事件在数据库 **版本号变化** 时触发。
- 这里在 `users` 表中增加了 `ageIndex` 索引，而不会影响原有数据。

删除旧的 objectStore 并迁移数据

如果需要**彻底更改数据结构**（如 `keyPath` 变更），需要：

1. **创建新 objectStore**，将旧数据迁移过去；
2. **删除原有 objectStore**。

示例：

```
const request = indexedDB.open("MyDatabase", 3); // 版本升级

request.onupgradeneeded = function (event) {
  const db = event.target.result;

  // 先删除旧的 objectStore
  if (db.objectStoreNames.contains("users")) {
    db.deleteObjectStore("users");
  }

  // 创建新的 objectStore
  const store = db.createObjectStore("newUsers", { keyPath: "userId" });
  store.createIndex("emailIndex", "email", { unique: true });
};
```

◆ 适用于：

- 变更 `keyPath`。
- 重新设计数据结构。

4. 面试回答示例

面试官：浏览器有哪些存储方式？适用场景是什么？

✓ 回答：

浏览器提供多种存储方式：

- **Cookie** (~4KB)：用于**身份验证、会话管理**，每次请求都会携带。
- **LocalStorage** (~5MB)：用于**存储用户偏好设置**，数据长期有效。
- **SessionStorage** (~5MB)：用于**单个页面会话数据**，关闭 Tab 即失效。
- **IndexedDB** (不限容量)：用于**存储大规模数据**，如**离线缓存、PWA 应用**。
- **Cache API**：用于**静态资源缓存**，加速页面加载。

面试官：IndexedDB 如何进行数据结构更新？

✓ 回答：

IndexedDB 不能直接修改表结构，只能通过**升级数据库版本** (`indexedDB.open(dbName, newVersion)`) 进行更改：

- **增加字段/索引**：在 `onupgradeneeded` 事件中，使用 `store.createIndex()` 增加索引。

- **删除表并迁移数据**: 先 `db.deleteObjectStore()` 删除旧表, 再 `db.createObjectStore()` 创建新表。
- **版本控制**: 每次变更结构, **数据库版本号必须递增**。

示例:

```
const request = indexedDB.open("MyDB", 2);
request.onupgradeneeded = (event) => {
  const db = event.target.result;
  const store = event.target.transaction.objectStore("users");
  if (!store.indexNames.contains("emailIndex")) {
    store.createIndex("emailIndex", "email", { unique: true });
  }
};
```

这样可以在 **数据库版本升级时** 新增 `emailIndex`, 实现数据结构更新。

5. 总结

- 存储方式选择:
 - 小型数据: `LocalStorage` / `SessionStorage`
 - 身份认证: `Cookie`
 - 结构化数据: `IndexedDB`
 - 资源缓存: `Cache API`
- IndexedDB 数据结构更新:
 - **增加索引**: 使用 `store.createIndex()`
 - **删除表并迁移**: 使用 `db.deleteObjectStore()`
 - **必须通过** `onupgradeneeded` **事件升级版本**

面试前端开发时, 回答要清晰、简洁, 同时展现你的理解和实战经验。下面是针对你的问题的系统回答, 并包含 `Promise.all()` 的手写实现。

美团1

1. 常用的 npm / pnpm / yarn 命令

npm (Node Package Manager)

- `npm init -y`: 初始化 `package.json`
- `npm install <package>`: 安装依赖
- `npm install -g <package>`: 全局安装依赖
- `npm uninstall <package>`: 卸载依赖
- `npm update`: 更新所有依赖
- `npm run <script>`: 运行 `package.json` 里定义的脚本

pnpm (性能优化的 npm)

- `pnpm install`: 安装依赖 (去重、提升性能)
- `pnpm add <package>`: 安装指定包
- `pnpm remove <package>`: 删除依赖
- `pnpm update`: 更新依赖

yarn (Facebook 开发)

- `yarn init -y`: 初始化项目
- `yarn add <package>`: 安装依赖
- `yarn remove <package>`: 删除依赖
- `yarn install`: 安装所有依赖

🔗 区别:

- `pnpm` **性能最佳**, 使用硬链接节省空间
- `yarn` **速度快**, 有更好的锁定机制
- `npm` **官方工具**, 新版也有类似 `yarn` 的特性

2. 了解 RESTful 规则吗?

RESTful 是一种 **API 设计风格**, 常见规则:

- **使用 HTTP 方法**: `GET` (获取)、`POST` (创建)、`PUT/PATCH` (更新)、`DELETE` (删除)
- **使用资源路径**: 例如 `/users/1` 表示获取 ID 为 1 的用户
- **无状态性**: 每个请求都是独立的
- **返回状态码**:
 - `200 OK`: 请求成功
 - `201 Created`: 资源创建成功
 - `400 Bad Request`: 请求错误
 - `401 Unauthorized`: 未授权
 - `404 Not Found`: 资源不存在
 - `500 Internal Server Error`: 服务器错误

3. CORS (跨域资源共享)

什么是跨域?

同源策略 (Same-Origin Policy, SOP) 限制了不同域名的网页间的资源访问。如果 `协议`、`域名`、`端口` 其中之一不同, 就会跨域。

CORS 解决方案

服务器通过 **设置 HTTP 头** 允许跨域:

```
res.setHeader("Access-Control-Allow-Origin", "*"); // 允许所有域访问
res.setHeader("Access-Control-Allow-Methods", "GET, POST, PUT, DELETE,
OPTIONS");
res.setHeader("Access-Control-Allow-Headers", "Content-Type, Authorization");
```

如果是 **复杂请求** (如 PUT/DELETE) , 浏览器会先发送 **预检请求** (OPTIONS) , 服务器需要允许它。

4. cookie 是如何实现的?

cookie 是存储在浏览器中的小型数据, 可用于身份验证、状态保持等。

设置 cookie

服务器返回 `Set-Cookie` 头:

```
res.setHeader("Set-Cookie", "token=abc123; HttpOnly; Secure");
```

- `HttpOnly`: 防止 JavaScript 访问 (防止 XSS 攻击)
- `Secure`: 仅限 HTTPS 传输
- `Max-Age`: cookie 过期时间

前端获取 cookie

```
document.cookie;
```

5. localStorage 与 cookie 的区别

特性	localStorage	cookie
存储大小	5MB	4KB
是否随请求发送	否	是
过期时间	需手动管理	可设置
适用场景	存储用户数据	身份认证

6. Promise.all() vs Promise.race()

Promise.all()

- **所有 Promise 都成功** 才会返回 **数组结果**
- **任何一个失败**, 整体就失败

```
Promise.all([p1, p2, p3])
  .then(res => console.log(res))
  .catch(err => console.log(err));
```

Promise.race()

- **返回最先完成的 Promise**
- **无论成功或失败**, 都会立即返回

```
Promise.race([p1, p2, p3])
  .then(res => console.log(res))
  .catch(err => console.log(err));
```

7. 说一下 Event Loop（事件循环）

JavaScript 是单线程的，执行顺序：

1. **同步代码** 先执行
2. 微任务（Microtask）
3. 宏任务（Macrotask）

示例：

```
console.log(1);
setTimeout(() => console.log(2), 0);
Promise.resolve().then(() => console.log(3));
console.log(4);
```

执行结果：1 → 4 → 3 → 2

在前端 JavaScript 中，**任务调度（Task Scheduling）** 分为 **宏任务（Macro Task）** 和 **微任务（Micro Task）**，它们的调度机制决定了 JavaScript 的异步执行顺序。

常见的宏任务：

宏任务类型	触发方式
<code>setTimeout / setInterval</code>	定时器触发
<code>setImmediate</code> (Node.js)	当前事件循环结束后执行
<code>requestAnimationFrame</code>	浏览器下一帧渲染前执行
I/O 任务 （文件操作、网络请求）	例如 <code>fetch</code> 的回调（注意 <code>fetch.then</code> 仍是微任务）
UI 渲染任务	例如 <code>DOMContentLoaded</code> 事件

常见的微任务：

微任务类型	触发方式
<code>Promise.then / catch / finally</code>	由 <code>Promise</code> 触发
<code>MutationObserver</code>	监听 DOM 变化
<code>queueMicrotask</code>	直接将任务放入微任务队列

8. Chrome 开发者工具使用过哪些

- **Elements**：修改 HTML/CSS
- **Console**：打印日志、调试 JS
- **Network**：查看 HTTP 请求、CORS
- **Sources**：断点调试

- **Performance**: 性能分析
- **Memory**: 内存泄漏检查
- **Application**: 查看 Cookie、LocalStorage、SessionStorage

9. flex 属性, flex: 1 具体指什么

```
flex: <flex-grow> <flex-shrink> <flex-basis>;
```

值	作用
flex-grow	伸展比例 (定义元素可增长的份数)
flex-shrink	收缩比例 (定义元素在空间不足时的缩小比例)
flex-basis	初始尺寸 (定义元素在分配剩余空间前的初始大小)

flex: 1 具体作用

1 flex-grow: 1 (扩展)

- 当父容器内 **有剩余空间** 时, flex-grow: 1 让该元素 **按比例填充剩余空间**。
- 如果多个元素都有 flex-grow: 1, 它们会 **等比例** 分配可用空间。

2 flex-shrink: 1 (收缩)

- 当父容器 **空间不足** 时, flex-shrink: 1 允许元素 **按比例缩小**。
- 如果所有子元素的 flex-shrink 值相同, 它们会 **等比例缩小**。

3 flex-basis: 0% (初始大小)

- 设置 flex-basis: 0%, 表示**元素的初始大小为 0**, 然后按 flex-grow 比例分配剩余空间。
- 如果 flex-basis 设为 auto, 元素会 **先根据内容确定大小**, 然后再按 flex-grow 伸展。

10. 手写 Promise.all()

```
function myPromiseAll(promises) {
  return new Promise((resolve, reject) => {
    let results = [];
    let completed = 0;
    if (promises.length === 0) resolve([]);

    promises.forEach((promise, index) => {
      Promise.resolve(promise)
        .then(value => {
          results[index] = value;
          completed++;
          if (completed === promises.length) {
            resolve(results);
          }
        })
        .catch(reject);
    });
  });
}
```

```
});
});
}

// 测试
const p1 = Promise.resolve(1);
const p2 = Promise.resolve(2);
const p3 = new Promise((resolve) => setTimeout(() => resolve(3), 1000));

myPromiseAll([p1, p2, p3]).then(console.log).catch(console.error);
```

核心逻辑:

- `Promise.resolve(promise)` 处理非 `Promise` 类型
- 用 `completed` 计数, 全部完成后 `resolve`
- 遇到 `reject` 立即返回错误

美团2

1. 浏览器的渲染过程

浏览器的渲染过程主要包含以下步骤:

1. 解析 HTML, 构建 DOM 树
 - HTML 被解析成 **DOM (Document Object Model)**, 形成层级结构。
2. 解析 CSS, 构建 CSSOM
 - CSS 被解析成 **CSSOM (CSS Object Model)**, 形成样式规则树。
3. 合并 DOM 和 CSSOM, 构建 Render Tree
 - 这一步会排除 `display: none` 的元素, 但 `visibility: hidden` 的元素仍会出现在 Render Tree。
4. 布局 (Layout)
 - 计算每个元素的大小和位置, 生成布局信息。
5. 绘制 (Painting)
 - 将每个元素的颜色、边框、阴影等绘制到屏幕上。
6. 合成 (Compositing)
 - 将不同的图层进行合成, 最终显示到屏幕上。

2. 浏览器的进程和线程

(1) 浏览器的多进程架构

现代浏览器 (如 Chrome) 采用 **多进程** 结构, 常见进程包括:

- **主进程** (Browser Process): 负责窗口管理、标签页管理、地址栏、网络请求等。
- **渲染进程** (Renderer Process): 负责 HTML、CSS、JavaScript 解析及渲染 (每个标签页一个)。
- **GPU 进程** (GPU Process): 负责页面合成、硬件加速。
- **网络进程** (Network Process): 负责网络请求, 如 HTTP、WebSocket。

(2) 渲染进程的线程

渲染进程内部包含多个线程：

线程	作用
GUI 渲染线程	负责 HTML/CSS 渲染、布局、绘制
JS 引擎线程	负责解析和执行 JavaScript（单线程）
事件触发线程	负责管理 setTimeout、setInterval、事件监听
定时器线程	负责 setTimeout、setInterval 的定时管理
网络线程	处理 XMLHttpRequest、Fetch、WebSocket 请求

3. 浏览器渲染触发的两个事件

(1) DOMContentLoaded 事件

- 触发时机：**DOM 解析完成**，但 **CSS/图片/iframe 等资源可能未加载完毕**。
- 适用于：希望在 DOM 加载完毕后执行 JavaScript（不依赖图片、CSS 加载）。

(2) load 事件

- 触发时机：**所有资源（包括 CSS、图片、视频、iframe）都加载完毕**。
- 适用于：确保所有外部资源加载完毕后执行操作。

4. Cookie 设置的几种方式

(1) 前端 JavaScript 设置

```
document.cookie = "username=John; expires=Fri, 31 Dec 2025 23:59:59 GMT; path=/";
```

- expires：设置过期时间
- path：限制 Cookie 的作用范围

(2) 后端 Set-Cookie 响应头

```
Set-Cookie: sessionId=abc123; HttpOnly; Secure; SameSite=Strict
```

- HttpOnly：前端 JS 不能访问，防止 XSS 攻击。
- Secure：仅 HTTPS 传输。
- SameSite=Strict：防止跨站请求伪造（CSRF）。

(3) 前端 document.cookie 获取

```
console.log(document.cookie); // username=John
```

5. Cookie 和 Session 的区别与联系

对比项	Cookie	Session
存储位置	客户端（浏览器）	服务器端
存储方式	文本数据	服务器内存/数据库
安全性	较低，容易被窃取	更安全（存储在服务器端）
生命周期	可设置 expires 过期时间	一般会话关闭后失效
适用场景	适用于存储少量数据，如用户偏好	适用于存储敏感数据，如登录状态

联系：Session 依赖 Cookie 存储 session_id，以便服务器识别用户。

6. 进程和线程的区别

对比项	进程	线程
定义	运行中的程序实例	进程内的执行单元
独立性	进程间数据独立	线程共享进程资源
通信方式	进程间通信（IPC）	共享内存，开销小
切换开销	较大（需要系统调度）	较小（同一进程内部切换）

7. 线程之间如何影响，如何防止影响

(1) 线程之间的影响

- 共享资源可能会发生竞争（Race Condition）
- 可能出现死锁（Deadlock）

(2) 如何防止影响

- 互斥锁（Mutex）：确保多个线程不会同时修改同一资源。
- 读写锁（Read-Write Lock）：允许多个线程读取，但写操作必须独占。
- 信号量（Semaphore）：限制线程的并发数量。
- 线程池（Thread Pool）：管理线程数量，避免资源耗尽。

8. HTTPS 握手过程

HTTPS 采用 TLS/SSL 进行加密，握手过程如下：

(1) 客户端发起请求

Client → Server: ClientHello

- 客户端发送 **ClientHello**，包含支持的 TLS 版本、加密算法等。

(2) 服务器响应

Server → Client: ServerHello

- 服务器发送 **ServerHello**，选择 TLS 版本和加密算法。
- 服务器提供 **SSL 证书**，包含公钥。

(3) 客户端验证证书

- 验证服务器的 SSL 证书是否合法（是否被 CA 机构签发）。
- 生成 **会话密钥 (Session Key)** 并用服务器公钥加密。

(4) 服务器解密会话密钥

- 服务器使用 **私钥解密** 出会话密钥。

(5) 双方使用会话密钥加密通信

- 之后所有数据都使用 **对称加密 (AES)** 进行安全通信。

美团3

1. 怎么学的前端？

我主要通过 **项目实战 + 文档学习 + 开源代码阅读** 来学习前端：

1. 基础阶段：

- 通过 **MDN 文档** 学习 HTML、CSS、JavaScript 基础。
- 做一些 **小项目**（如静态页面、动态表单）来熟悉基本技术。

2. 框架阶段：

- 学习 **Vue** 和 **React**，结合官方文档和示例项目进行实践。
- 研究组件化开发、状态管理（Vuex/Redux）、路由等核心概念。

3. 进阶提升：

- 阅读 **开源项目源码**，如 Vue、React、Webpack 等，理解其实现原理。
- 研究 **前端工程化**（如脚手架、CI/CD、自动化构建等）。
- 参与前端项目，解决实际开发中的问题。

2. 对前端的哪些方向感兴趣？

我主要关注 **前端工程化、性能优化和前端架构**：

- **前端工程化**：脚手架开发、自动化构建（Webpack/Vite）、Monorepo、CI/CD。
- **性能优化**：首屏优化、懒加载、SSR、PWA、Tree-shaking。
- **前端架构**：微前端、前端模块化、Serverless 方向。

3. 讲讲你做的脚手架，怎么开发的？怎么发布的？

Vite 是一个现代化的构建工具，专注于为开发者提供快速的启动和构建体验。它主要是通过利用浏览器的原生模块（ESM）来提供极速的开发体验，而构建时使用了高效的打包工具（如 esbuild）来处理和优化代码。Vite 对前端开发工具链进行了优化，极大地提高了开发效率。

1. 安装和初始化 Vite 项目

首先，你需要初始化一个 Vite 项目。可以使用 `npm`、`yarn` 或 `pnpm` 来创建一个新的 Vite 项目。

```
# 使用 npm 创建 vite 项目
npm create vite@latest my-vite-project --template vue
cd my-vite-project
npm install
```

2. 定义脚手架功能

你需要确定脚手架提供的功能，可能包括：

- 项目初始化：提供不同的模板（如 React、Vue、Svelte 等）。
- 插件支持：例如集成 CSS 预处理器、ESLint、PostCSS 等。
- 配置文件管理：提供常见的配置选项，帮助用户快速配置项目。
- 模板生成：自动生成一些常用的文件和目录结构。

3. 自定义配置

根据项目需求，可以定制 Vite 配置文件。Vite 提供了非常灵活的配置选项，你可以在 `vite.config.js` 中做如下配置：

- 配置 `base` 路径、端口、代理等。
- 配置插件，比如 Vue 或 React 插件。
- 配置打包选项、路径别名、环境变量等。

例如：

```
// vite.config.js
import { defineConfig } from 'vite';
import vue from '@vitejs/plugin-vue';

export default defineConfig({
  plugins: [vue()],
  server: {
    port: 3000,
    proxy: {
      '/api': 'http://localhost:5000'
    }
  }
});
```

4. 添加常用功能

- **插件支持**：你可以为项目集成一些常用的 Vite 插件，比如 `vite-plugin-pages` 来自动生成路由，或是 `vite-plugin-vue2` 来支持 Vue 2。
- **自动化命令**：根据脚手架的需求，你可以在 `package.json` 中设置自定义的命令（比如 `init` 或 `build`）。

```
{
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "serve": "vite preview",
    "init": "node scripts/init.js"
  }
}
```

5. 项目模板创建

你可以设计模板，将公共的代码和配置项抽离成模板文件。例如，可以创建一个 `template` 文件夹，其中包含基础的文件结构（`index.html`、`App.vue` 或 `index.js`、`package.json` 等）。

6. 测试脚手架

测试是确保脚手架正常工作的重要步骤。你可以使用一些工具（如 Jest 或 Mocha）进行单元测试和集成测试，确保项目功能正常。

7. 发布脚手架

开发完成后，你可以将 Vite 脚手架发布到 npm 上，方便其他开发者使用。

- 初始化 npm 包：

```
npm init
```

- 修改 `package.json` 配置，包括名称、版本、描述、入口文件、脚本命令等。
- 登录 npm 账号并发布：

```
npm login
npm publish
```

发布后，其他开发者就可以通过以下命令使用你的脚手架：

```
npm install -g your-vite-cli
```

8. 维护和更新

发布后的脚手架可能会随着时间的推移和技术栈的变化而更新。你需要定期更新你的脚手架，修复 bug，并增加新的功能。

Vite 发布的原理与优化

Vite 的核心特点之一是利用现代浏览器的 ESM 模块特性，这使得它在开发阶段具有非常快速的热更新和构建体验。在构建阶段，Vite 使用 `esbuild` 进行打包和优化，它是用 Go 语言编写的，速度非常快。

- **开发阶段：**Vite 不会对整个项目进行打包，而是通过原生的 ESM 支持将模块直接传递到浏览器，提升了启动速度。
 - **构建阶段：**当运行 `vite build` 时，Vite 会将项目构建成优化后的静态文件，并对资源进行代码分割、压缩和优化。
-

4. Babel 能做什么？具体能转义哪些方面？

Babel 是一个 **JavaScript 编译器**，主要用于：

- **ES6+ 代码转换**：如箭头函数、模板字符串、`let/const` 等转为 ES5。
- **JSX 转换**：React 组件中的 JSX 转为 `React.createElement` 形式。
- **TypeScript 转换**：将 TypeScript 代码转换为 JavaScript。
- **Polyfill 处理**：借助 `@babel/polyfill` 添加兼容性代码（如 `Promise`、`Array.prototype.includes`）。
- **Tree Shaking**：去除未使用的代码，减少打包体积。

5. 对 React 和 Vue 的看法

对比项	React	Vue
架构	函数式编程思想	MVVM 模型
数据流	单向数据流， <code>useState</code>	双向绑定 (<code>v-model</code>)
状态管理	Redux、Recoil、Context	Vuex、Pinia
虚拟 DOM	运行时 diff	预编译 + 运行时 diff
适用场景	复杂交互、多状态管理	中小型项目，快速开发

个人看法：

- **React** 更适合大型复杂应用，生态丰富，Hooks 非常灵活。
- **Vue** 上手简单，适合快速开发，单文件组件（SFC）体验好。

6. 页面加载时间长的判断和优化

判断

1. Chrome DevTools - Performance
 - 录制性能日志，分析 **关键阻塞阶段**（HTML 解析、CSS 计算、JS 执行、网络请求）。
2. Network 面板
 - 观察资源加载时间，找出 **耗时长的请求**（如图片、脚本）。

优化方案

- **减少 HTTP 请求**：合并 CSS/JS，使用雪碧图。
- **懒加载**：使用 `loading="lazy"` 延迟加载图片，按需加载组件。
- **缓存策略**：使用 `Cache-Control`、`Service Worker` 提高资源复用率。
- **SSR/SSG**：减少客户端渲染时间，提高首屏速度。
- **优化 JS**：减少 `setTimeout/setInterval` 影响，减少 `Repaint/Reflow`。

7. HTML 是怎么解析的？

1. **构建 DOM 树**：HTML 解析器解析标签，创建 DOM 结构。
 2. **构建 CSSOM**：解析 CSS 规则，生成样式表。
 3. **合并 DOM + CSSOM**：形成渲染树（Render Tree）。
 4. **布局计算**：计算每个元素的位置、大小。
 5. **绘制（Painting）**：渲染元素到屏幕。
-

8. 讲讲 JavaScript 的原型链

原型链 是 JavaScript 继承的核心机制：

- 每个对象都有一个 `__proto__`，指向其 **构造函数的 prototype**。
- 访问对象属性时，如果当前对象没有，会沿着 `__proto__` 逐级查找，直到 `null`。

示例：

```
function Person(name) {
  this.name = name;
}
Person.prototype.sayHello = function() {
  console.log(`Hello, ${this.name}`);
};

const p = new Person("Alice");
p.sayHello(); // 找不到 -> 查找 prototype -> 找到方法
```

9. JavaScript 的继承方式

1. **原型链继承**
 2. **构造函数继承**
 3. **组合继承（原型链 + 构造函数）**
 4. **ES6 class 继承**
 5. **Object.create 继承**
-

10. 代码题

(1) 实现 `instanceof`

```
function myInstanceOf(left, right) {
  let proto = Object.getPrototypeOf(left);
  while (proto) {
    if (proto === right.prototype) return true;
    proto = Object.getPrototypeOf(proto);
  }
  return false;
}

// 测试
console.log(myInstanceOf([], Array)); // true
console.log(myInstanceOf({}, Object)); // true
console.log(myInstanceOf([], Object)); // true
```

```
console.log(myInstanceOf(123, Number)); // false
```

(2) 实现 findLastIndex

```
Array.prototype.myFindLastIndex = function(callback) {  
  for (let i = this.length - 1; i >= 0; i--) {  
    if (callback(this[i], i, this)) return i;  
  }  
  return -1;  
};  
  
// 测试  
const arr = [1, 2, 3, 4, 5, 3];  
console.log(arr.myFindLastIndex(x => x === 3)); // 5  
console.log(arr.myFindLastIndex(x => x > 4)); // 4
```

美团4

1. 常见 HTTP 状态码

状态码	含义
2xx 成功	
200	请求成功
201	资源创建成功
3xx 重定向	
301	永久重定向
302	临时重定向
304	资源未修改（缓存）
4xx 客户端错误	
400	请求错误
401	未授权（未登录/Token 失效）
403	禁止访问
404	资源未找到
5xx 服务器错误	
500	服务器内部错误
502	网关错误
503	服务器不可用

2. JWT 的优缺点

✓ 优点

1. **无状态**：不需要存储会话信息，适合分布式系统。
2. **安全性高**：使用 **HMAC/RS256** 进行签名，防篡改。
3. **携带用户信息**：Token 本身可存储用户 ID、角色等信息。

✗ 缺点

1. **无法主动失效**：一旦签发，无法手动注销，需等待过期。
2. **体积较大**：Token 可能比 Session ID 更大，影响性能。
3. **泄露后不可撤销**：若 JWT 被盗，攻击者可用到期。

4. 如何隐藏一个元素？

方法	是否回流	是否重绘	影响布局
<code>display: none;</code>	✓ 回流	✓ 重绘	✓ 影响
<code>visibility: hidden;</code>	✗ 不回流	✓ 重绘	✗ 不影响
<code>opacity: 0;</code>	✗ 不回流	✓ 重绘	✗ 不影响
<code>position: absolute; left: -9999px;</code>	✗ 不回流	✗ 不重绘	✓ 影响
<code>transform: scale(0);</code>	✗ 不回流	✓ 重绘	✗ 不影响

5. 如何判断数据类型？

```
console.log(typeof null); // "object" (历史遗留问题)
console.log(typeof []); // "object"
console.log(typeof {}); // "object"
console.log(typeof 123); // "number"
console.log(typeof 'abc'); // "string"
console.log(typeof true); // "boolean"
console.log(typeof function(){}); // "function"
console.log(typeof Symbol()); // "symbol"
console.log(typeof undefined); // "undefined"
```

方法	适用场景
<code>typeof</code>	基本数据类型，不能区分 <code>null</code> 、 <code>array</code>
<code>instanceof</code>	判断引用类型，如 <code>arr instanceof Array</code>
<code>Object.prototype.toString.call(obj)</code>	<code>"[object Type]"</code> 精准判断
<code>Array.isArray(obj)</code>	判断是否为数组
<code>isNaN(value)</code>	判断是否为 NaN

6. `toString()` 的原理

- **普通对象**：默认调用 `Object.prototype.toString`，返回 `"[object Object]"`。
- **数组**：返回元素拼接后的字符串，如 `[1,2,3].toString()` 返回 `"1,2,3"`。
- **函数**：返回代码字符串。
- **重写 `toString`**：

```
const obj = {
  toString() {
    return "自定义 toString";
  }
};
console.log(obj.toString()); // "自定义 toString"
```

7. Webpack 的作用

- **模块化打包**：支持 `ES6 Modules`、`CommonJS`、`AMD` 等。
- **代码分割**：按需加载 `lazy loading`，减少首屏加载时间。
- **优化性能**：Tree Shaking、压缩、图片优化等。

8. Vite 为什么快？

- **基于 ES Modules**：不需要 `bundle`，直接使用浏览器加载。
- **按需编译**：Webpack 需要打包所有模块，Vite 只编译使用的模块。
- **原生支持 TS/JSX**：基于 `esbuild`，比 `Babel` 快 **20~30 倍**。

9. TypeScript 的作用

- **静态类型检查**，提高代码可维护性。
- **自动补全、代码提示、类型推导**，提高开发效率。
- **减少运行时错误**。

10. `let` 和 `const` 的作用

- `let` 允许变量重新赋值，`const` 不能修改引用（但对象属性可变）。
- `let` 和 `const` 都 **不存在变量提升**，并且 **有块级作用域**。
- `let` 允许声明后赋值，`const` 必须在声明时赋值。

11. Promise API

方法	作用
<code>Promise.resolve(value)</code>	返回成功的 Promise
<code>Promise.reject(error)</code>	返回失败的 Promise
<code>Promise.all(promises)</code>	并行执行，所有成功才返回，任意失败即失败
<code>Promise.race(promises)</code>	返回第一个执行完成的 Promise（无论成功失败）
<code>Promise.allSettled(promises)</code>	返回所有结果（无论成功失败）
<code>Promise.any(promises)</code>	返回第一个成功的 Promise，全部失败才失败

12. 移动端适配方案

1. 媒体查询 (`@media`)。
2. rem/em 适配 (`html` 根元素动态计算 `font-size`)。
3. vw/vh 适配 (视口单位, `vw = viewport width`)。
4. flex 布局 (自动适应屏幕大小)。

13. 代码题

判断回文数字

方法 1: 字符串反转

```
function isPalindrome(num) {  
  const str = num.toString();  
  return str === str.split('').reverse().join('');  
}  
console.log(isPalindrome(121)); // true  
console.log(isPalindrome(123)); // false
```

方法 2: 双指针

```
function isPalindrome(num) {  
  const str = num.toString();  
  let left = 0, right = str.length - 1;  
  while (left < right) {  
    if (str[left] !== str[right]) return false;  
    left++;  
    right--;  
  }  
  return true;  
}
```

方法 3：数学方法（不使用字符串）

```
function isPalindrome(num) {
  if (num < 0 || (num % 10 === 0 && num !== 0)) return false;
  let reversed = 0, original = num;
  while (num > 0) {
    reversed = reversed * 10 + num % 10;
    num = Math.floor(num / 10);
  }
  return reversed === original;
}
```

美团5

1. BFC（Block Formatting Context）是什么？

BFC 是 **块级格式化上下文**，是一个容器，在该容器内，元素的布局规则受到一定的控制。BFC 对外部元素的布局不会产生影响，但容器内的元素之间的布局会互相影响。BFC 主要用于解决清除浮动、避免元素重叠等问题。

如何触发 BFC：

- 根元素或 `<html>` 元素。
- 浮动元素（`float` 不为 `none`）。
- `position` 为 `absolute` 或 `fixed` 的元素。
- `display: flow-root`。
- `display: table`、`display: inline-block`、`display: flex` 等。

BFC 的作用：

- 清除浮动。
- 阻止外边距合并（margin collapsing）。

2. 如何实现居中？

水平居中：

- 通过 `text-align`（适用于行内元素）：

```
.parent {
  text-align: center;
}
.child {
  display: inline-block;
}
```

- 通过 `margin: auto`（适用于块级元素）：

```
.parent {
  width: 100%;
}
.child {
  width: 200px;
  margin: 0 auto;
}
```

- 通过 flexbox:

```
.parent {
  display: flex;
  justify-content: center;
}
.child {
  width: 200px;
}
```

垂直居中:

- 通过 flexbox:

```
.parent {
  display: flex;
  align-items: center;
  justify-content: center;
}
.child {
  width: 200px;
  height: 200px;
}
```

- 通过 position:

```
.parent {
  position: relative;
  height: 500px;
}
.child {
  position: absolute;
  top: 50%;
  left: 50%;
  transform: translate(-50%, -50%);
}
```

3. 写 CSS 遇到的问题

- **BFC 问题**: 浮动元素引起的父元素高度塌陷, 可以通过给父元素添加 `overflow: hidden` 或 `clearfix` 来解决。
- **定位问题**: `position` 的不同值对布局产生影响, 了解 `relative`、`absolute`、`fixed` 和 `sticky` 的区别。
- **边距合并**: 块级元素的垂直边距合并 (`margin collapse`) 现象, 可通过设置 `padding`、`border` 或 `overflow` 来避免。

4. `async` 和 `defer` 的区别

- `async` :

- 脚本在 HTML 解析过程中异步加载，加载完成后立即执行。
- 会阻塞后续的 HTML 解析，因此不推荐与 HTML 解析同时进行。
- 适用于脚本不依赖于其他脚本，且不依赖 DOM 渲染的场景。

```
<script async src="script.js"></script>
```

- `defer` :

- 脚本会在 HTML 解析完成后执行。
- 保证脚本按照在文档中出现的顺序执行。
- 适用于需要 DOM 渲染后执行的脚本。

```
<script defer src="script.js"></script>
```

5. 浏览器怎么解析 HTML?

浏览器解析 HTML 的过程大致如下：

1. **词法分析 (Lexical Analysis)**：将 HTML 文件转换为一系列 **tokens**。
 2. **构建 DOM 树**：浏览器解析 HTML 代码并生成 DOM（文档对象模型）树，DOM 树是浏览器与页面交互的核心。
 3. **构建 CSSOM 树**：解析 CSS 样式并与 DOM 树合并，形成 CSSOM（CSS 对象模型）树。
 4. **渲染树构建**：将 DOM 树与 CSSOM 树合并，构建渲染树，渲染树包含了样式与布局信息。
 5. **布局计算 (Layout)**：根据渲染树计算每个元素的位置与大小。
 6. **绘制 (Painting)**：最终绘制元素到屏幕上。
-

6. 块级元素、行内元素、行内块元素的区别

- **块级元素 (Block Element)** :

- 独占一行，宽度默认 100%。
- 可以设置 `width` 和 `height`。
- 例如：`<div>`，`<p>`，`<h1>`。

- **行内元素 (Inline Element)** :

- 不独占一行，宽度由内容决定。
- 不能设置 `width` 和 `height`。
- 例如：`<span>`，`<a>`，`<strong>`。

- **行内块元素 (Inline-block Element)** :

- 不独占一行，可以设置 `width` 和 `height`，同时与行内元素并排显示。
 - 例如：`<img>`，`<button>`，`<input>`。
-

7. `proto` 和 `prototype` 的区别

- `prototype`：是所有 JavaScript 对象的属性，指向对象的原型。每个构造函数都有一个 `prototype` 属性。

- `__proto__`：是实例对象的属性，指向实例的原型链。每个对象都有一个 `__proto__` 属性，指向它的构造函数的 `prototype` 属性。

示例：

```
function Person() {}  
const person = new Person();  
console.log(person.__proto__ === Person.prototype); // true  
console.log(Person.prototype.__proto__ === Object.prototype); // true
```

8. 代码题

深拷贝

```
function deepClone(obj) {  
  if (obj === null || typeof obj !== 'object') return obj;  
  let clone = Array.isArray(obj) ? [] : {};  
  for (let key in obj) {  
    if (obj.hasOwnProperty(key)) {  
      clone[key] = deepClone(obj[key]);  
    }  
  }  
  return clone;  
}  
  
const obj = { a: 1, b: { c: 2 } };  
const clonedObj = deepClone(obj);  
console.log(clonedObj); // { a: 1, b: { c: 2 } }
```

千位分隔符

```
function formatNumber(num) {  
  return num.toString().replace(/\B(?=(\d{3})+(?!\d))/g, ',');  
}  
  
console.log(formatNumber(123456789)); // "123,456,789"
```

箭头函数 `this` 指向问题

```
const obj = {  
  name: 'Alice',  
  greet: function() {  
    setTimeout(() => {  
      console.log(this.name); // "Alice"  
    }, 1000);  
  }  
};  
obj.greet();
```

箭头函数不会创建自己的 `this`，它会继承外部函数的 `this`。

判断一颗二叉树是否对称

```
function isSymmetric(root) {
  if (!root) return true;

  function isMirror(t1, t2) {
    if (!t1 && !t2) return true;
    if (!t1 || !t2) return false;
    return (t1.val === t2.val) &&
      isMirror(t1.left, t2.right) &&
      isMirror(t1.right, t2.left);
  }

  return isMirror(root.left, root.right);
}
```

腾讯1

Q1.项目遇到的难点，怎么解决的？

Q2.Vite 与 Webpack 详细对比

开发模式不同：

- **Vite**：在开发环境中，Vite 利用浏览器原生支持的 ES 模块特性，直接启动开发服务器，然后按需编译依赖文件。当浏览器请求某个模块时，Vite 才对其进行实时编译，实现了真正的按需编译，减少了不必要的编译工作，尤其是在大型项目中优势明显。
- **Webpack**：Webpack 在启动开发服务器前，需要先对整个项目进行打包，分析模块依赖、编译并构建出一个浏览器可识别的打包文件，这个过程会消耗较多时间，特别是对于大型项目，文件数量和依赖关系复杂，打包时间会很长。

底层语言差异：

- **Vite**：Vite 使用 Esbuild 进行预构建依赖，Esbuild 是用 Go 语言编写的。Go 语言具有高效的编译和执行速度，在纳秒级别，能快速完成依赖的预构建，相比之下，Vite 在打包速度上比 Webpack 有显著提升。
- **Webpack**：Webpack 是基于 Node.js 构建的，Node.js 是毫秒级别的，在处理大型项目的复杂依赖关系和大量文件时，速度相对较慢。

热更新机制优化：

- **Vite**：Vite 的热模块替换（HMR）速度快，当某个模块内容改变时，只需要让浏览器重新请求该模块即可，无需重新打包整个应用，能提供近乎即时的反馈，开发者修改代码后能快速看到效果，提高开发效率。
- **Webpack**：Webpack 虽然也支持热模块替换，但在更新时需要重新构建发生变化的模块及其相关依赖，当项目规模较大时，这个过程可能会比较耗时，导致热更新速度慢，影响开发体验。

构建结果优化：

- **Vite**：Vite 在生产环境下使用 Rollup 作为打包工具，Rollup 在处理 ES 模块时表现出色，能利用 Tree Shaking 等优化技术，确保最终构建的代码更精简，体积更小，从而提高加载和执行效率。
- **Webpack**：Webpack 构建出的文件往往相对较大，虽然可以通过各种插件进行优化，但配置复杂，且优化效果可能不如 Vite 明显，尤其是在处理大型项目时，可能会生成一些冗余代码，影响加

载速度。

配置复杂度不同：

- **Vite**：Vite 的默认配置能满足大多数开发需求，配置简单明了，开发者可以快速上手，将更多精力放在代码本身，而不是复杂的配置上，这也间接提高了开发效率。
- **Webpack**：Webpack 功能强大，但配置复杂，随着项目规模的增大，配置文件可能会变得庞大且难以维护，开发者需要花费大量时间和精力去调整配置，以达到理想的构建效果，这在一定程度上影响了开发速度。

对比项	Vite	Webpack
启动速度	🚀 极快（毫秒级）	🐢 慢（秒级，特别是大型项目）
热更新（HMR）	快速、精准，仅更新变更模块	更新缓慢，常常刷新整个模块树
构建方式	原生 ES 模块（ESM）+ 现代浏览器	把所有代码打包成 bundle 文件
打包工具	使用 <code>esbuild</code> （开发）+ <code>rollup</code> （生产）	使用自身的打包流程
配置难度	开箱即用，配置简单	配置项丰富，但复杂、冗长
生态插件	插件数量快速增长，和 rollup 通用	插件生态庞大、成熟
构建产物体积	较小（使用 Rollup）	较大（tree-shaking 效果有限）
支持框架	Vue3 原生支持，React、Svelte 等都有模板	几乎支持所有框架
TypeScript 支持	原生支持，无需额外 loader	需手动配置 ts-loader 或 babel
文档体验	简洁直观、现代化	信息全面但略显冗长

1.核心理念区别

✔ Vite 的理念：原生模块开发 + 极速构建

- 利用现代浏览器支持 ESM，开发时无需打包，直接基于模块加载。
- 使用 `esbuild`（Go 编写）进行极速转译，远快于 babel。
- 生产环境打包使用 `Rollup`，提供更小更优化的包体积。

✔ Webpack 的理念：构建一切的模块打包器

- 构建过程中将所有资源（JS、CSS、图片等）统一打包。
- 强调自定义能力：可以控制每一个 loader、plugin、构建流程。
- 为旧浏览器兼容做了很多底层支持。

2.性能对比 (以 Vue/React 项目为例)

操作	Vite	Webpack
初次启动	0.3s	3~8s
修改代码热更新	100~300ms	1~2s
打包构建	与 Webpack 相近或更快	略慢，构建时间取决于配置
构建体积	更小 (Rollup 优化好)	可能更大 (插件多)

3.示例对比：一个简单项目的配置

 Webpack 项目基本配置：

```
module.exports = {
  entry: './src/index.js',
  output: {
    filename: 'bundle.js',
    path: __dirname + '/dist',
  },
  module: {
    rules: [
      { test: /\.js$/, use: 'babel-loader' },
      { test: /\.css$/, use: ['style-loader', 'css-loader'] }
    ]
  },
  plugins: [new HtmlWebpackPlugin()],
};
```

 Vite 项目配置：

```
// vite.config.ts
import { defineConfig } from 'vite';
import vue from '@vitejs/plugin-vue';

export default defineConfig({
  plugins: [vue()],
  server: {
    port: 3000
  }
});
```

4.使用场景

场景	推荐工具
Vue3 项目 / 追求极速开发体验	✓ Vite
小型项目 / 快速原型验证	✓ Vite
老项目维护 / 多年积累的 Webpack 项目	✓ Webpack
需要高度自定义构建流程	✓ Webpack
构建 npm 库 / 组件库	✓ Vite 或 Rollup

Q3. vue3响应式布局的原理

Vue 3 响应式系统通过 **Proxy + WeakMap + effect 副作用函数** 实现了“数据驱动视图”的核心理念，从而让开发者只需关注数据，布局自然就响应式地改变了。

Vue 的响应式布局指的通常是：**当数据变化时，视图会自动更新，布局跟着变化。**

比如页面上根据 `v-if` 或 `v-show` 控制元素显示与隐藏、或根据数据动态渲染 DOM 的位置与内容。Vue 3 正是通过上述响应式系统实现：

- 数据变了，组件重新渲染（通过虚拟 DOM 和 diff）。
- 布局也随之变化，达到响应式效果。

Q4. 为什么选择typescript?

TypeScript 是 JavaScript 的超集，它为 JavaScript 添加了 **静态类型系统** 和 **更强的开发工具支持**。越来越多的前端项目（包括 Vue3、React、Angular）都在采用 TypeScript，是现代前端开发的大趋势。

✓ 一、提升代码的健壮性和可维护性

◆ 静态类型检查，开发期发现错误

- JavaScript 是动态类型语言，变量类型不明确，容易在运行时出错。
- TypeScript 在编译期就能发现错误，如类型不匹配、未定义变量、函数参数不一致等。

```
function sum(a: number, b: number): number {  
  return a + b;  
}  
sum(1, "2"); // ✗ 报错：类型不匹配
```

✓ 二、增强 IDE 智能提示和开发效率

◆ 自动补全、参数提示、跳转到定义、重构等体验极佳

- TypeScript 让 IDE（如 VSCode）像“懂代码一样”辅助你写代码。

```
interface User {  
  name: string;  
  age: number;  
}  
const u: User = { name: "Tom", age: 25 };  
u.name; // 自动提示属性名
```

✓ 三、提升团队协作效率

◆ 明确接口定义，减少沟通成本

- 使用接口（interface）或类型别名（type）可以将数据结构定义清晰、统一，利于多人协作。

```
interface LoginParams {
  username: string;
  password: string;
}
function login(params: LoginParams) {
  // ...
}
```

◆ API 和组件之间的输入输出更清晰，避免“猜参数”

✓ 四、增强大型项目的可扩展性与可维护性

◆ 便于模块化管理和代码重构

- 对变量、函数、组件都有清晰的类型约束，IDE 重构时能智能跟踪所有依赖项。

◆ 避免“踩坑式开发”，降低项目后期维护成本

- 特别适合中大型项目或多人协作项目。

✓ 五、更好地支持主流框架和生态

框架	TS 支持情况
Vue3	Composition API 深度集成 TypeScript
React	Hooks 全面支持 TS 泛型
Angular	天生基于 TypeScript 编写
Node.js	使用 TS 构建服务端逻辑更安全

此外，常见的构建工具（Vite、Webpack）、UI 组件库（Ant Design、Element Plus）、后端 API SDK 等也都提供了完善的 TypeScript 类型定义。

✓ 六、更强的类型系统（可选但强大）

- 支持泛型、联合类型、类型推导、条件类型等高级用法：

```
function identity<T>(arg: T): T {
  return arg;
}
```

- 可以实现复杂的类型逻辑，如接口继承、映射类型、工具类型（Partial、Pick、Omit 等）

基本类型相关扩展

1. 元组 (Tuple)

JS 里只有数组，数组元素的类型可以随意混合，且长度不固定。而 TS 的元组类型明确规定了元素的数量和每个位置元素的类型。

```
// 定义一个元组类型，第一个元素是 string 类型，第二个元素是 number 类型
let person: [string, number] = ['John', 30];
```

2. 枚举 (Enum)

JS 没有内置的枚举类型，而 TS 提供了枚举类型，用于定义一组命名的常量。

```
// 定义一个枚举类型 Direction
enum Direction {
  Up = 1,
  Down,
  Left,
  Right
}
let myDirection: Direction = Direction.Up;
```

高级类型

1. 交叉类型 (Intersection Types)

交叉类型能把多个类型合并成一个类型，新类型包含所有类型的特性。

```
interface Person {
  name: string;
}
interface Employee {
  employeeId: number;
}
// 定义一个交叉类型，包含 Person 和 Employee 的所有属性
type PersonEmployee = Person & Employee;
let personEmp: PersonEmployee = { name: 'Alice', employeeId: 123 };
```

2. 联合类型 (Union Types)

联合类型表示一个值可以是几种类型之一。

```
// 定义一个联合类型，值可以是 string 或 number
let value: string | number;
value = 'hello';
value = 123;
```

3. 类型别名 (Type Aliases)

类型别名可以给一个类型起一个新名字，方便复用复杂类型。

```
// 定义一个类型别名，用于表示 string 或 number 类型
type StringOrNumber = string | number;
let data: StringOrNumber = 'abc';
```

4. 字面量类型 (Literal Types)

字面量类型允许你指定一个值必须是某个具体的字符串、数字或布尔值。

```
// 定义一个字面量类型，值只能是 'success' 或 'error'
type Status = 'success' | 'error';
let currentStatus: Status = 'success';
```

5. 可选链类型 (Optional Chaining)

虽然 ES2020 引入了可选链操作符，但在 TS 里它与类型系统结合得更紧密，可进行类型检查。

```
interface User {
  address?: {
    street?: string;
  };
}
let user: User = {};
// 使用可选链操作符访问属性，TS 会进行类型检查
let street = user.address?.street;
```

6. 空值合并类型 (Nullish Coalescing)

和可选链类似，ES2020 引入了空值合并操作符，在 TS 中可结合类型系统确保类型安全。

```
let value: string | null = null;
// 使用空值合并操作符，若 value 为 null 或 undefined，则使用 'default'
let result = value ?? 'default';
```

函数相关类型

1. 函数重载 (Function Overloading)

TS 支持函数重载，允许为一个函数提供多个不同的调用签名。

```
// 函数重载签名
function add(a: number, b: number): number;
function add(a: string, b: string): string;
// 函数实现
function add(a: any, b: any): any {
  return a + b;
}
let numResult = add(1, 2);
let strResult = add('hello', ' world');
```

这些额外的类型系统特性让 TS 具有更强的类型检查能力，能在编译阶段发现很多潜在的错误，提高代码的可维护性和健壮性。

☑ 七、渐进式使用，兼容老项目

- TypeScript 可以逐步引入，现有的 JS 文件也能通过 `JSDoc` 和类型声明获取类型支持。

腾讯2

Q1: HTTP 基于什么协议？

✔ HTTP 是基于 TCP 协议的应用层协议。

- HTTP 本质是“请求-响应”协议，运行在 **应用层**。
- TCP 处于 **传输层**，提供了可靠、面向连接的数据传输服务。
- 所以每次 HTTP 请求都建立在一次 **TCP 连接** 之上（通常是 3 次握手 + 请求 + 响应 + 4 次挥手）。

Q2: HTTP 可不可以基于 UDP？

理论上可以，但实际中不可行/不推荐。

特性	TCP	UDP
是否可靠传输	✔ 是（有序 + 校验 + 重传）	✗ 否（丢包/乱序）
是否面向连接	✔	✗
是否支持流控制	✔	✗
延迟	相对较高	很低
应用场景	HTTP、HTTPS、FTP、SSH 等	视频流、DNS、语音通话等

⚠ 所以为什么 HTTP 不用 UDP？

- HTTP 需要完整且可靠地传输数据（比如 HTML、JSON）。
- UDP 不可靠、不按顺序、不校验，无法满足 HTTP 请求-响应机制。
- 但可以用于对延迟要求极高、不怕丢包的应用（如视频直播）。

Q3: HTTPS 是什么？

✔ HTTPS = HTTP + SSL/TLS

- HTTPS 是“加密的 HTTP”协议，它在 **HTTP 和 TCP** 之间插入了一个加密层——**TLS（或早期的 SSL）**。
- 目的是：
 - 加密传输内容，防止被窃听
 - 校证书身份，防止伪装
 - 保证数据完整性，防止篡改

Q4: TLS 是什么？

✔ TLS（Transport Layer Security）：传输层安全协议，用于加密网络通信。

- 是 HTTPS 的核心加密协议（SSL 的继任者，SSL 现已废弃）。
- 提供三大安全功能：
 1. **加密**：对称加密（AES）、非对称加密（RSA、ECC）、密钥协商（DH）

- 2. **认证**：通过数字证书验证服务器身份（如百度的 HTTPS 网站有证书）
- 3. **完整性校验**：使用哈希算法（如 SHA256）防止数据被篡改

Q5: 如何避免中间人攻击（Man-in-the-Middle Attack）？

✔ 使用 HTTPS（TLS）是最有效方式，防止中间人攻击。

👉 中间人攻击是这样发生的：

你 --> 中间人（伪装成服务器） --> 真服务器

中间人可能：

- 拦截你发的请求
- 篡改你返回的数据
- 偷看你的账号密码等敏感信息

✔ 如何防止：

防护方法	说明
TLS 加密通信	数据全程加密，即使被拦截也看不懂
数字证书验证身份	服务器证书中包含其公钥及认证机构签名，浏览器可验证真伪
浏览器校验证书链	如发现假证书或证书失效会主动报警（“此网站不安全”）
HSTS（严格传输安全）	强制使用 HTTPS，防止降级攻击（如被劫持成 HTTP）
不随便忽略证书警告	用户点击“继续访问”很可能就掉坑了

🧠 总结一张图理解：



Q6: TCP阻塞控制

TCP 的阻塞控制实际上是拥塞控制的一部分，它通过动态调整拥塞窗口（cwnd）来控制发送速率，防止网络出现拥堵。整个机制包括慢启动、拥塞避免、快重传和快恢复四种算法。刚开始采用慢启动快速增大拥塞窗口，当达到阈值后切换为拥塞避免；如果出现丢包，通过快重传迅速恢复丢包，并结合快恢复策略避免重回慢启动，从而实现更平稳的传输过程。

✓ 1. TCP 有什么特点？

- 面向连接、可靠传输
- 通过**流量控制**和**拥塞控制**机制保障传输稳定性

✓ 2. 拥塞控制 VS 流量控制

	拥塞控制	流量控制
控制目标	避免网络堵塞（比如路由器爆了）	避免接收方被“写爆”
控制对象	整个网络	TCP 的接收方
变量名	<code>cwnd</code> （拥塞窗口）	<code>rwnd</code> （接收窗口）
实现机制	4 大算法	滑动窗口协议

✓ 3. 拥塞控制的四种算法（核心）

① 慢启动（Slow Start）

- 初始 `cwnd = 1 MSS`，指数级增长（每收到一个 ACK，`cwnd += 1`）
- 直到达到慢启动门限（`ssthresh`）后，切换到拥塞避免
- 避免一开始就把网络冲垮

② 拥塞避免（Congestion Avoidance）

- `cwnd` 线性增长（每收到一个 ACK，`cwnd += 1/cwnd`）
- 稳定提升传输速率，防止拥塞

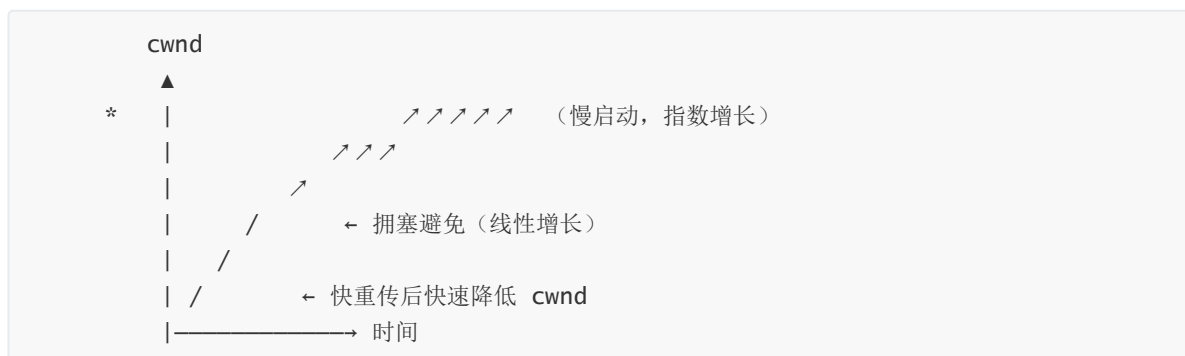
③ 快重传（Fast Retransmit）

- 连续收到 **三个重复 ACK**，认为该数据包可能丢失，**立即重传**，不用等超时

④ 快恢复（Fast Recovery）

- 重传后不进入慢启动，而是将 `cwnd` 降低一半，然后继续拥塞避免
- 比“从头开始”更高效

✓ 4. 结合图示理解拥塞窗口 `cwnd`（可以这样补充）：



Q1: 自我介绍

Q2: 平时在学校的成绩怎么样?

Q3: 讲一下TCP的三次握手与四次挥手详细过程

一、三次握手 (Three-way Handshake)

三次握手是为了解决客户端和服务端建立可靠连接时的同步与确认问题。

📌 场景: 客户端要与服务器建立连接

🔄 第一次握手: 客户端 → 服务端

- 客户端发送一个 **SYN (同步) 报文段**, 表示要建立连接。
 - 报文中:
 - `SYN = 1`
 - `seq = x` (客户端初始序号)
 - 状态: 客户端进入 `SYN_SEND` 状态
-

🔄 第二次握手: 服务端 → 客户端

- 服务端收到后, 如果同意连接, 回复一个 **SYN+ACK 报文段**
 - 报文中:
 - `SYN = 1, ACK = 1`
 - `ack = x + 1` (确认客户端的序号)
 - `seq = y` (服务端初始序号)
 - 状态: 服务端进入 `SYN_RECEIVED` 状态
-

🔄 第三次握手: 客户端 → 服务端

- 客户端收到服务端的 `SYN+ACK` 报文后, 发送一个 **ACK 报文段**
 - 报文中:
 - `ACK = 1`
 - `ack = y + 1` (确认服务端的序号)
 - 状态: 客户端进入 `ESTABLISHED` 状态, 连接建立完成
 - 服务端收到 ACK 后也进入 `ESTABLISHED` 状态
-

✅ 连接正式建立, 双方可以开始数据传输。

👋 二、四次挥手 (Four-way Handshake)

TCP 连接断开时需要 4 次挥手, 是为了实现双方的“半关闭”, 确保数据能完整传输完。

✂ 第一次挥手：客户端 → 服务端

- 客户端主动关闭，发送一个 **FIN 报文段**
 - 报文中：
 - `FIN = 1`
 - `seq = u`
 - 状态：客户端进入 `FIN_WAIT_1` 状态
-

✂ 第二次挥手：服务端 → 客户端

- 服务端收到 FIN 后，发送 **ACK 报文段**
 - 报文中：
 - `ACK = 1`
 - `ack = u + 1`
 - 状态：服务端进入 `CLOSE_WAIT`，客户端进入 `FIN_WAIT_2`
-

✂ 第三次挥手：服务端 → 客户端

- 服务端确认没数据要发了，发送 **FIN 报文段**
 - 报文中：
 - `FIN = 1`
 - `seq = v`
 - 状态：服务端进入 `LAST_ACK`
-

✂ 第四次挥手：客户端 → 服务端

- 客户端收到 FIN 后，发送 **ACK 报文段**
 - 报文中：
 - `ACK = 1`
 - `ack = v + 1`
 - 状态：客户端进入 `TIME_WAIT`，等待 2 倍 MSL 后彻底关闭
 - 服务端收到 ACK 后进入 `CLOSED` 状态
-

✓ 至此，TCP 连接完全关闭。

Q4：为什么是三次握手不是两次或者四次？为什么需要第四次挥手？

❓ 为什么不能只用两次？

如果是两次：

- 假设客户端发送了 SYN，服务端回复 ACK，但客户端**没有收到 ACK**，认为连接失败；
 - 但服务端其实已经进入连接状态，后续客户端再发 SYN 会导致混乱；
 - **第三次握手就是为了客户端确认服务端已准备好通信，双方状态同步。**
-

❓ 为什么不用四次？

- 三次已经可以满足“双方都知道对方可以发送和接收”的前提；
- 第三次客户端发的 ACK，不仅确认了服务端的 SYN，还告诉服务端：“我也准备好了”；
- 所以第四次没必要，多一次是冗余。

❓ 为什么需要第四次挥手？

- **保证连接的正常终止**：TCP 连接的关闭需要双方都进行确认和操作。第四次挥手是整个关闭过程的最后一步，它确保了客户端和服务端都完成了各自的关闭操作，并且都知道对方已经准备好关闭连接。只有当双方都进入 CLOSE 状态后，TCP 连接才被真正关闭，相关的资源（如端口号、缓冲区等）才能被释放。
- **确认服务器的关闭请求**：在第三次挥手时，服务器向客户端发送了 FIN 报文，请求关闭连接。客户端收到这个 FIN 报文后，需要发送 ACK 应答报文来确认收到了服务器的关闭请求。这个 ACK 报文是客户端对服务器关闭连接请求的确认，让服务器知道客户端已经准备好关闭连接。如果没有第四次挥手，服务器就无法确定客户端是否收到了它发送的 FIN 报文，**可能会导致服务器一直等待客户端的响应，从而无法正确释放资源。**

Q5：讲一下HTTP各个版本以及特性

① HTTP/0.9 (1991) ——「最早版本」

- 只支持 GET 方法；
- 不支持请求头、响应头、状态码；
- 只能传输 HTML 页面；
- 建立连接 → 返回文件 → 立即关闭（无持久连接）。

特点：极其简陋，仅用于超文本传输。

② HTTP/1.0 (1996)

- 支持 GET、POST、HEAD 方法；
- 引入了请求头、响应头；
- 支持传输图片、音频等非文本内容；
- 默认使用**短连接**（每个请求都要建立一次TCP连接）；
- 缺乏 Host 头（后来补充支持）。

特点：初具雏形，但连接效率较低。

③ HTTP/1.1 (1997，现阶段主流)

- 默认启用**长连接**（Connection: keep-alive）；
- 支持**请求管道化**（Pipelining）；
- 引入 Host 头，支持虚拟主机；
- 支持分块传输（Transfer-Encoding: chunked）；
- 增强缓存控制（Cache-Control、ETag 等）；
- 更丰富的状态码（如 100 Continue）。

特点：稳定、全面，仍是大多数网站使用的版本。

④ HTTP/2 (2015)

- 基于二进制协议，不再是纯文本格式；
- 多路复用：一个连接上可并发多个请求，解决队头阻塞；
- 头部压缩 (HPACK)：减少重复的头部信息传输；
- 支持服务端推送 (Server Push)；
- 一般基于 HTTPS + TLS 部署。

特点：效率高、性能强、用户体验提升明显。

⑤ HTTP/3 (2022, 基于 QUIC 协议)

- 底层改为 UDP + QUIC，替代传统 TCP；
- 彻底解决 TCP 队头阻塞问题；
- 握手更快，集成了 TLS 1.3；
- 仍支持多路复用、头部压缩 (QPACK)、服务端推送；
- 更适合移动网络、弱网环境。

特点：传输速度更快、更安全、更稳定，是未来主流方向。

✓ 各版本对比表

特性	HTTP/1.0	HTTP/1.1	HTTP/2	HTTP/3
长连接	✗	✓	✓	✓
多路复用	✗	⚠ (有限)	✓	✓
二进制传输	✗	✗	✓	✓
头部压缩	✗	✗	✓ (HPACK)	✓ (QPACK)
服务端推送	✗	✗	✓	✓
连接协议	TCP	TCP	TCP	UDP (QUIC)
队头阻塞问题	严重	有	有 (流内)	✗

Q6: node.js常用的有哪些模块？

Node.js 内置模块如 `fs`、`http`、`path` 等用于文件、网络、路径处理，而 `express`、`mongoose`、`axios` 等第三方模块则提升开发效率、支持数据库、接口通信和鉴权等常见场景。

✓ 一、Node.js 常用的 内置模块（原生模块）

内置模块不需要安装，可以直接 `require` 使用。

1. `fs` (File System) —— 文件系统模块

- 功能：读写文件、创建目录、删除文件等。
- 使用场景：文件上传、读取配置、日志系统。

```
const fs = require('fs');
fs.readFile('example.txt', 'utf-8', (err, data) => {
  if (err) throw err;
  console.log(data);
});
```

2. http / https —— 创建服务端或请求客户端

- **功能：**创建 HTTP 服务器或客户端。
- **使用场景：**构建 API 服务、代理服务器等。

```
const http = require('http');
http.createServer((req, res) => {
  res.writeHead(200, {'Content-Type': 'text/plain'});
  res.end('Hello, world!');
}).listen(3000);
```

3. path —— 路径处理模块

- **功能：**拼接、解析、格式化路径，兼容不同操作系统。
- **使用场景：**构建静态路径、处理上传路径等。

```
const path = require('path');
const fullPath = path.join(__dirname, 'public', 'index.html');
console.log(fullPath);
```

4. os —— 操作系统信息模块

- **功能：**获取 CPU、内存、平台、主机名等系统信息。
- **使用场景：**系统监控、环境判断。

```
const os = require('os');
console.log(os.platform()); // win32, linux, darwin 等
console.log(os.totalmem());
```

5. url —— URL 解析模块

- **功能：**解析 URL 字符串为对象。
- **使用场景：**处理请求链接、路由解析。

```
const url = require('url');
const parsed = url.parse('http://example.com/index?name=jack', true);
console.log(parsed.query.name); // 输出 jack
```

6. events —— 事件处理模块

- **功能：**发布订阅模式的核心模块。
- **使用场景：**自定义事件、模块解耦。

```
const EventEmitter = require('events');
const emitter = new EventEmitter();

emitter.on('data', (msg) => console.log('收到:', msg));
emitter.emit('data', 'Hello');
```

7. stream —— 流操作模块

- **功能：**按块处理数据（读写流），适合大文件或实时数据。
- **使用场景：**文件上传下载、大文件处理。

```
const fs = require('fs');
const rs = fs.createReadStream('bigfile.txt');
rs.on('data', chunk => console.log(chunk.toString()));
```

8. crypto —— 加密模块

- **功能：**加密、解密、签名、哈希等。
- **使用场景：**密码校验、JWT、数据加密。

```
const crypto = require('crypto');
const hash = crypto.createHash('sha256').update('hello').digest('hex');
console.log(hash); // 输出 SHA-256 的哈希值
```

9. child_process —— 创建子进程

- **功能：**在 Node 中运行 shell 命令或其他脚本。
- **使用场景：**调用 Python 脚本、构建工具、自动部署。

```
const { exec } = require('child_process');
exec('ls', (err, stdout, stderr) => {
  if (err) return console.error(err);
  console.log(stdout);
});
```

10. util —— 工具类模块

- **功能：**常用工具函数，如 `promisify`、继承等。
- **使用场景：**回调转 Promise、调试格式化等。

```
const util = require('util');
const fs = require('fs');
const readFile = util.promisify(fs.readFile);
readFile('test.txt', 'utf8').then(console.log);
```

✓ 二、Node.js 常用的 第三方模块 (npm 包)

这些模块需要用 `npm install` 安装。

模块	用途	示例
<code>express</code>	Web服务器框架	构建 RESTful API
<code>axios</code>	发起 HTTP 请求	代替原生 http 更简洁
<code>mongoose</code>	MongoDB ORM	操作数据库更方便
<code>sequelize</code>	MySQL ORM	定义模型+关系映射
<code>jsonwebtoken</code>	JWT 认证	登录鉴权
<code>dotenv</code>	读取环境变量	<code>.env</code> 配置文件
<code>cors</code>	跨域处理中间件	配合 express 使用
<code>socket.io</code>	实时通信框架	聊天系统、推送服务
<code>nodemon</code>	自动重启开发工具	保存代码自动重启服务器

Q7: v8引擎如何进行垃圾内存回收?

- **标记 - 清除 (Mark - Sweep) 算法:**
 - **标记阶段:** 从根对象 (全局对象、调用栈等) 开始, 遍历对象之间的引用关系, 标记所有可到达 (即被引用) 的对象。
 - **清除阶段:** 扫描堆内存, 清除 (释放) 未被标记的对象所占用的内存空间。但这种方式可能会产生内存碎片。
- **标记 - 整理 (Mark - Compact) 算法:** 在标记 - 清除的基础上改进。标记阶段和标记 - 清除一样, 标记可到达对象。整理阶段会将存活的对象向一端移动, 然后直接清理掉边界以外的内存, 解决了内存碎片问题, 但移动对象有一定性能开销。
- **分代回收 (Generational Collection) :** V8 将堆内存分为新生代和老生代。
 - **新生代:** 存放存活时间短的对象, 采用复制算法。当新生代空间快满时, 将存活对象复制到另一个空间, 同时清理原空间。因为新生代对象存活时间短, 复制成本相对较低。
 - **老生代:** 存放存活时间长的对象, 使用标记 - 清除或标记 - 整理算法。由于老生代对象存活时间长, 复制成本高, 所以采用更适合的这两种算法。

Q8: 平时怎么学习前端的?

Q9: `Promise.all`、`Promise.race` 和 `Promise.allSettled`

✓ 1. `Promise.all()`

功能：

- 接受一个 `Promise` 数组（或类数组对象），返回一个新的 `Promise`。
- 只有当所有的 `Promise` 都成功时，返回的 `Promise` 才会 `resolve`，否则只要有一个 `Promise` 失败，它就会 `reject`。

使用场景：

- 用于在所有异步操作完成后继续执行某些操作，并且要求所有操作成功。

示例：

```
const promise1 = Promise.resolve(3);
const promise2 = Promise.resolve(4);
const promise3 = new Promise((resolve, reject) => setTimeout(reject, 100, 'error'));

Promise.all([promise1, promise2, promise3])
  .then(values => {
    console.log(values); // 不会执行
  })
  .catch(error => {
    console.log(error); // 'error', 因为有一个 Promise 失败
  });
```

特点：

- 返回：一个 `Promise`，其 `resolve` 时带有一个数组，数组包含每个输入 `Promise` 的 `resolve` 值。
- 失败条件：只要有任何一个 `Promise` 失败，整个 `Promise.all()` 会 `reject`，并且返回的 `error` 是第一个失败的 `Promise` 的错误。

✓ 2. `Promise.race()`

功能：

- 接受一个 `Promise` 数组（或类数组对象），返回一个新的 `Promise`。
- 返回的 `Promise` 会采用第一个完成的 `Promise` 的结果，无论是 `resolve` 还是 `reject`。

使用场景：

- 用于在多个并发操作中，只关注第一个完成的操作（比如超时控制）。

示例：

```
const promise1 = new Promise((resolve, reject) => setTimeout(resolve, 100, 'one'));
const promise2 = new Promise((resolve, reject) => setTimeout(resolve, 500, 'two'));
const promise3 = new Promise((resolve, reject) => setTimeout(reject, 200, 'three'));

Promise.race([promise1, promise2, promise3])
  .then(value => {
    console.log(value); // 'one', 第一个 resolve 的 Promise
  })
  .catch(error => {
    console.log(error); // 'three', 第一个 reject 的 Promise
  });
```

特点:

- **返回:** 一个 `Promise`，其 `resolve` 或 `reject` 时带有第一个完成的 `Promise` 的结果（无论成功或失败）。
- **失败条件:** 如果第一个完成的是 `reject`，返回的 `Promise` 也会 `reject`。

✓ 3. `Promise.allSettled()`

功能:

- 接受一个 `Promise` 数组（或类数组对象），返回一个新的 `Promise`。
- **无论每个 `Promise` 是 `resolve` 还是 `reject`，返回的 `Promise` 会在所有 `Promise` 完成时 `resolve`，并且返回每个 `Promise` 的最终状态（`fulfilled` 或 `rejected`）以及对应的结果。**

使用场景:

- 用于处理多个异步操作，且不关心每个操作是否成功，只关心它们是否完成（例如，统计操作结果）。

示例:

```
const promise1 = Promise.resolve(3);
const promise2 = new Promise((resolve, reject) => setTimeout(reject, 100, 'error'));
const promise3 = Promise.resolve(5);

Promise.allSettled([promise1, promise2, promise3])
  .then(results => {
    console.log(results);
    // [
    //   { status: 'fulfilled', value: 3 },
    //   { status: 'rejected', reason: 'error' },
    //   { status: 'fulfilled', value: 5 }
    // ]
  });
```

特点：

- **返回：** 一个 `Promise`，其 `resolve` 时带有一个结果数组，数组中每个元素都有两个属性：
 - `status`： `"fulfilled"` 或 `"rejected"`。
 - `value` 或 `reason`： 如果 `status` 是 `fulfilled`，则是 `value`；如果是 `rejected`，则是 `reason`。
- **失败条件：** 不管任何 `Promise` 是成功还是失败，`Promise.allSettled()` 总是会返回一个 `resolve` 的 `Promise`，只是 `value` 或 `reason` 可能不同。

 总结区别：

特性/方法	<code>Promise.all</code>	<code>Promise.race</code>	<code>Promise.allSettled</code>
输入	<code>Promise</code> 数组	<code>Promise</code> 数组	<code>Promise</code> 数组
返回值	返回一个 <code>Promise</code> ，结果是数组，按顺序对应输入的 <code>Promise</code> 结果	返回一个 <code>Promise</code> ，结果是第一个完成的 <code>Promise</code> （成功或失败）	返回一个 <code>Promise</code> ，结果是每个 <code>Promise</code> 的状态和对应的结果
成功条件	所有 <code>Promise</code> 都成功，返回值是它们的 <code>resolve</code> 结果	第一个完成的 <code>Promise</code> ，无论是 <code>resolve</code> 还是 <code>reject</code>	所有 <code>Promise</code> 都完成，不管是成功还是失败
失败条件	只要有任何一个 <code>Promise</code> 失败， <code>reject</code>	第一个完成的 <code>Promise</code> 失败， <code>reject</code>	不管是否成功，都不会 <code>reject</code> ，只有状态信息
常见用途	并行处理多个任务，要求所有任务成功	只关心第一个完成的任务（如超时控制）	不关心每个任务是否成功，关心任务是否完成

腾讯4

Q1： 你都有哪些优化性能的手段？ 如何衡量性能提升？

1. 性能优化手段（分为加载阶段 & 运行阶段）

◆ 加载阶段：

1. **代码拆分（Code Splitting）**： 按需加载路由/模块/组件
2. **懒加载（Lazy Load）**： 图片、视频、非首屏资源懒加载
3. **资源压缩与合并：**
 - JS/CSS压缩（Terser/CSSNano）
 - HTML 压缩
 - Gzip 或 Brotli 传输压缩

- 4. **CDN 分发资源**：减少 RTT，提高加载速度
- 5. **缓存策略优化**：
 - HTTP 缓存（强缓存/协商缓存）
 - Service Worker 离线缓存
- 6. **图片优化**：
 - WebP 格式
 - 懒加载
 - 响应式图片（srcset）
- 7. **Preload/Prefetch**：预加载关键资源，加快下一步加载

◆ **运行阶段：**

- 1. **虚拟列表**：减少 DOM 渲染，适用于长列表场景
- 2. **防抖 / 节流**：优化频繁事件监听
- 3. **异步加载数据**：非关键数据延后请求
- 4. **合并 DOM 操作**，减少重排重绘
- 5. **Web Worker 多线程处理**：避免主线程卡顿
- 6. **合理使用 requestIdleCallback**：低优先级任务异步执行

2. 性能衡量方式

工具 / 方法	指标
Chrome DevTools - Performance 面板	渲染帧率、Recalculate Style、Repaint、Layout 时间等
Lighthouse	FCP、LCP、TTI、TBT、CLS 等性能指标
Web Vitals (web-vitals SDK)	LCP、FID、CLS等真实用户体验
Performance API (performance.now())	精准测量某段 JS 执行耗时
自定义打点（埋点 + BI）	页面加载、请求耗时、白屏时间等

Q2：如何监测一个电商首页的平均首屏耗时（考虑不同用户设备）？

实现方式：

- 1. **接入 Web Vitals 库（推荐）**
使用 `web-vitals` SDK 采集 LCP、FCP、FID 指标

```
import { getLCP } from 'web-vitals';
getLCP(console.log); // 上传 BI 接口即可
```

- 2. **埋点统计（自定义）：**
 - 页面开始加载：`performance.timing.navigationStart`
 - 首屏元素加载完毕：某 DOM 出现或图片 `onload`
 - `首屏时间 = 元素出现时间 - navigationStart`
- 3. **上传至日志系统（如 ELK + Grafana、阿里云日志服务）**

- 日志字段包括：userId、设备类型、机型、时间戳、首屏耗时
- 后台做平均值分析、分设备类型分析

Q3：怎么测大盘的首屏耗时？怎么定义首屏？

💡 首屏定义：

用户打开页面后，第一屏完整显示的内容区域（不滚动）加载完成的时间点。

📌 如何定义哪些元素属于首屏：

- 通过 `window.innerHeight` 和 `getBoundingClientRect()` 判断是否在首屏范围
- 也可以通过打 `data-firstscreen` 属性标记关键 DOM 元素（如 Banner、搜索框、轮播图、前几个商品卡片）

```
const isInViewport = el => el.getBoundingClientRect().top < window.innerHeight;
```

📌 判断“首屏完成加载”：

- 所有“首屏关键元素”都触发 `onload` 或出现在 DOM 中
- 图片/video/异步请求完成也要考虑

Q4：如何更通用地定义首屏？LCP 怎么定义？

✅ LCP (Largest Contentful Paint) 定义：

页面中 最大可视内容元素渲染完成的时间，是衡量首屏加载体验最核心的指标。

- Chrome 默认自动识别：
 - `img` 标签
 - `video` 海报图
 - `block-level` 文本节点（如 `<h1>`、`<div>` 等）

✅ 如何通用监测首屏耗时？

- 使用 LCP + 自定义 DOM 监听结合：
 - LCP 作为通用首屏完成时间
 - 对业务关键元素做打点补充
- 利用 `PerformanceObserver` API 监听：

```
new PerformanceObserver((entryList) => {
  for (const entry of entryList.getEntries()) {
    console.log('LCP: ', entry);
  }
}).observe({ type: 'largest-contentful-paint', buffered: true });
```

Q5：懒加载的方式有哪些？图片/视频轮播如何优化？

常见懒加载手段：

技术	说明
<code>loading="lazy"</code> （原生）	Chrome/Safari/Edge 已支持，适合图片、iframe
IntersectionObserver	精准监听元素是否进入视口，实现懒加载
图片占位符（LQIP）	模糊小图占位，加载完成后替换清晰图
骨架屏	提升感知体验，减少“白屏”
异步组件加载	Vue/React 支持动态 <code>import()</code>
SSR + Hydration	服务端首屏渲染 + 客户端激活交互

针对电商轮播优化策略：

1. 首屏只加载首图/首视频，其余懒加载

```


```

2. IntersectionObserver 动态加载非可视图片

```
const observer = new IntersectionObserver((entries) => {
  entries.forEach(entry => {
    if (entry.isIntersecting) {
      entry.target.src = entry.target.dataset.src;
      observer.unobserve(entry.target);
    }
  });
});
observer.observe(document.querySelector('img[data-src]'));
```

3. 资源预加载：使用 `prefetch` 预加载下一张轮播图
4. 压缩图片、WebP、延迟加载视频文件

Q6：你常用的 Git 命令有哪些？

 基本操作

- 1. `git clone <repo-url>`：克隆远程仓库
- 2. `git status`：查看当前工作区状态
- 3. `git add . / <filename>`：添加改动到暂存区
- 4. `git commit -m "message"`：提交改动

分支管理

1. `git branch`: 查看本地分支
2. `git checkout -b <branch-name>`: 新建并切换到新分支
3. `git switch <branch>`: 切换分支 (推荐新写法)
4. `git merge <branch>`: 合并分支
5. `git rebase <branch>`: 变基操作, 线性化提交记录

远程操作

1. `git pull`: 拉取远程最新代码并合并
2. `git push origin <branch>`: 将本地分支推送到远程
3. `git fetch`: 拉取远程更新但不合并

历史操作

1. `git log`: 查看提交记录
2. `git diff`: 查看改动差异
3. `git reset --hard <commit>`: 回退到指定版本
4. `git reflog`: 查看历史 HEAD 变动, 常用于误操作恢复

常见工作流

- PR 工作流: `feature` → `dev` → `main`
- hotfix 修复 → `rebase` → `merge`

Q7: 除了代码层, 网络层有哪些性能优化方法?

网络层性能优化手段:

减少请求数量

1. 合并资源 (雪碧图、JS/CSS打包)
2. 使用 HTTP/2 (多路复用, 减少队头阻塞)
3. 使用 Service Worker 缓存资源并离线可用

缓存机制

1. HTTP 缓存
 - 强缓存 (Expires / Cache-Control)
 - 协商缓存 (ETag / Last-Modified)
2. DNS 预获取 (dns-prefetch)

```
<link rel="dns-prefetch" href="//cdn.example.com">
```

提升网络传输效率

1. 使用 Gzip 或 Brotli 压缩响应内容
2. 启用 TLS1.3 提升 HTTPS 性能
3. 使用 CDN 将资源下发至用户最近的边缘节点

✔ 延迟加载 & 异步加载

- JS: `<script async/defer>`
- CSS: 非关键 CSS 异步加载

✔ 图片资源优化

1. 使用 WebP 格式
2. 多尺寸图片匹配设备 (响应式图片 srcset)
3. 懒加载 (IntersectionObserver)

✔ 使用优质网络协议

- HTTP/3 (基于 QUIC, 降低丢包影响)
- 开启 Keep-Alive 减少 TCP 连接开销

Q8: CDN 是什么? 适用于哪些场景? 优劣势?

🔍 CDN (Content Delivery Network) 简介:

一种内容分发网络, 通过将资源部署到全球分布的边缘节点, 用户访问资源时可就近从最近的节点获取, 大大加快访问速度。

✔ 使用场景:

场景	说明
静态资源分发	JS/CSS/图片/字体等静态文件
视频/音频流媒体分发	CDN 支持大流量、多并发视频分发
分布式用户	跨地域访问, 提升全球用户访问速度
前端工程化项目	通常会构建发布到 CDN 目录, 配合缓存策略使用
高并发场景	CDN 分担主服务器压力, 提高可用性和稳定性

🕒 CDN 优势:

1. **加快加载速度**: 靠近用户的边缘节点返回资源, 减少 RTT
2. **减轻源站压力**: 边缘缓存资源降低主服务器负载
3. **高可用**: CDN 自动负载均衡、故障容灾
4. **抗攻击能力强**: 部分 CDN 提供 DDoS 保护

▼ CDN 局限性:

局限	说明
缓存更新延迟	一旦部署 CDN，资源更新需要处理版本号或清缓存
不适合频繁变更的内容	比如动态接口、用户定制化页面
可能增加调试复杂度	Debug 时不容易直观看到真实响应内容
边缘节点不稳定	部分小型 CDN 网络质量差，节点少

Q9：你在项目中有没有一些你觉得比较满意的技术点呢？

Q10：有没有了解过websocket？

当然，这是一种用于实现客户端和服务端之间双向通信的协议。

简介

WebSocket 是一种持久化协议，相较于 HTTP 的“请求-响应”模式，WebSocket 是全双工、实时通信协议，由 HTTP 升级握手建立连接。

特点

- 建立后长连接不断开
- 支持服务端主动推送消息到客户端
- 更适合实时场景（如：聊天室、股票行情、游戏联机）

使用场景

- IM 即时通讯
- 实时协同编辑（文档、代码）
- 实时图表、监控面板
- 在线客服系统

实现方式（前端示例）

```
const socket = new WebSocket("wss://example.com/socket");
socket.onmessage = (event) => {
  console.log("收到消息:", event.data);
};
socket.send("hello server");
```

Q11：了解跨域吗？为什么会产生跨域，怎么解决？

什么是跨域？

浏览器的同源策略（Same-Origin Policy）限制：协议、域名、端口三者必须一致，否则就会发生“跨域”。

例如：

- 你的前端页面来自 `http://example.com`
- 如果它请求 `http://api.example.com` 或 `https://example.com:8080`，就算跨域

🔍 为什么要有同源策略？

- 安全性考虑，防止 XSS / CSRF 等攻击
- 防止恶意脚本读取敏感信息（如 cookie）

✅ 常见跨域解决方案

方式	原理	说明
CORS (推荐)	后端设置响应头 <code>Access-Control-Allow-Origin</code>	支持复杂请求、支持认证、现代主流方式
JSONP	利用 <code><script></code> 标签不受同源限制	仅支持 GET 请求，已过时
代理转发	本地开发用 node/nginx 做接口代理	本质是“绕过”浏览器同源检测
WebSocket	不受同源策略限制	适合实时通信
iframe + postMessage	跨域通信方式之一	用于页面嵌套等场景
document.domain	修改同一主域下的子域通信	仅限二级域名间通信，已弃用

Q12：你能否让浏览器不去校验某次跨域请求？了解 CORS 的限制吗？

这个问题在问你对 CORS 机制的深入理解。

🔍 答案是：不能完全阻止浏览器校验。

浏览器是执行 CORS 的执行者，不是服务器。

- CORS 是由浏览器实现的安全策略，服务器只能“告诉”浏览器是否允许访问
- 服务端设置响应头，例如：

```
Access-Control-Allow-Origin: https://example.com
Access-Control-Allow-Credentials: true
```

才能允许跨域请求成功返回。

🔧 CORS 核心机制

- Preflight 预检请求 (OPTIONS)
 - 对复杂请求（带认证、PUT/DELETE、自定义头等）浏览器会自动发一个预检
- 关键头部说明

Header	说明
<code>Access-Control-Allow-Origin</code>	允许访问的来源
<code>Access-Control-Allow-Credentials</code>	是否允许携带 cookie
<code>Access-Control-Allow-Methods</code>	允许的请求方法
<code>Access-Control-Allow-Headers</code>	允许的自定义请求头

⊗ 注意：

- 浏览器不可能“忽略 CORS 校验”，只能服务端允许
- 设置 `mode: 'no-cors'` 只适合发无返回的请求，并不会绕开限制

Q13：TCP 跟 UDP 有什么区别？

项目	TCP	UDP
连接方式	面向连接（三次握手）	无连接
是否可靠	可靠（有确认机制）	不可靠（无确认）
是否有序	有序传输	不保证顺序
速度	慢（有重传、流控）	快（开销小）
场景	文件传输、网页请求	视频直播、语音、DNS

简单记忆：

- TCP 保证可靠、慢
- UDP 牺牲可靠、快

Q14：浏览器的事件循环有没有了解？

是的，事件循环（Event Loop）是 JavaScript 实现异步编程的核心机制。

🔄 执行流程：

1. 执行主线程同步代码（全栈入栈）
2. 遇到异步任务（如 `setTimeout`, `Promise`），交由浏览器或 Node 执行
3. 回调进入任务队列（宏任务/微任务）
4. 主线程空了以后，按顺序执行微任务 → 宏任务

📦 任务分类：

类型	举例	优先级
微任务（micro task）	<code>Promise.then</code> , <code>MutationObserver</code>	优先执行
宏任务（macro task）	<code>setTimeout</code> , <code>setInterval</code> , <code>requestAnimationFrame</code>	后执行

示例代码：

```
console.log('start');

setTimeout(() => console.log('setTimeout'), 0);

Promise.resolve().then(() => console.log('promise'));

console.log('end');

// 输出顺序: start → end → promise → setTimeout
```

Q15: node.js 怎么给前端返回一个 404 状态？

使用原生 `http` 模块返回状态码即可：

```
const http = require('http');

const server = http.createServer((req, res) => {
  if (req.url !== '/api') {
    res.statusCode = 404;
    res.setHeader('Content-Type', 'text/plain');
    res.end('404 Not Found');
  } else {
    res.statusCode = 200;
    res.end('Hello');
  }
});

server.listen(3000);
```

如果使用 `Express` 框架：

```
app.use((req, res) => {
  res.status(404).send('Not Found');
});
```

Q16: Node.js 哪些手段可以分析内存泄漏？

常见内存泄漏排查工具：

1. 内置工具

- `process.memoryUsage()`：查看内存占用
- `global.gc()`：配合 `--expose-gc` 手动触发垃圾回收

2. 调试工具

- **Chrome DevTools**：通过 `node --inspect` 远程调试
- **node-heapdump**：生成 `.heapsnapshot`，配合 Chrome 分析堆快照
- **clinic.js**：自动化诊断性能问题（内存、CPU、事件循环）

3. npm 工具

- `memwatch-next`、`leakage`：监听堆使用情况

🔗 常见内存泄漏来源：

- 全局变量未释放
- 闭包引用导致对象不能被回收
- 定时器/事件监听未清除
- 缓存（如 Map、对象池）无限增长

Q17: Map 跟 WeakMap 有什么区别？

特性	Map	WeakMap
键类型	任意类型	只能是对象
是否可枚举	是（ <code>forEach</code> 、 <code>keys</code> ）	否
是否可被垃圾回收	否，强引用	是，弱引用（无引用即回收）
场景	一般结构化数据存储	绑定 DOM、私有数据缓存

🔗 WeakMap 典型应用：

```
const wm = new WeakMap();
let dom = document.getElementById('myDiv');

wm.set(dom, { clicked: true });

// 当 dom 被移除，WeakMap 中的键值对自动释放，不造成内存泄漏
```

Q18: 箭头函数跟普通函数有什么区别？

箭头函数的 `this` 是从其定义时所在的函数作用域中继承的。

特性	普通函数	箭头函数
<code>this</code> 指向	动态，调用时决定	静态，定义时绑定外层作用域
是否能当构造函数	☑ 能（ <code>new</code> ）	✗ 不能
是否有 <code>arguments</code> 对象	☑ 有	✗ 没有
是否能用作方法	☑ 推荐	✗ 不推荐（ <code>this</code> 问题）

🔗 示例对比：

```
function normal() {
  console.log(this); // this 指调用者
}

const arrow = () => {
  console.log(this); // this 指向定义时外部作用域
};
```

✓ 什么时候用箭头函数？

- 更简洁的函数表达
- 保持 `this` 绑定（如回调函数中）

当然可以，下面是一个箭头函数保持 `this` 绑定的经典示例，说明为什么它在回调函数中非常适合使用：

✓ 使用箭头函数保持 `this` 绑定的场景示例

假设我们有一个对象，它内部使用了 `setTimeout` 来延迟输出 `name` 属性：

✗ 普通函数：`this` 丢失

```
const person = {
  name: 'Alice',
  sayHi: function () {
    const name = "lili";
    setTimeout(function () {
      const name = "zhangsan";
      console.log('Hi, I am ' + this.name); // ✗ this 指向 window 或 undefined
    }, 1000);
  }
};

person.sayHi(); // 输出: Hi, I am undefined
```

✓ 箭头函数：`this` 正确绑定到外层作用域

```
const person = {
  name: 'Alice',
  sayHi: function () {
    setTimeout(() => {
      console.log('Hi, I am ' + this.name); // ✓ this 指向 person 对象
    }, 1000);
  }
};

person.sayHi(); // 输出: Hi, I am Alice
```

🔗 解释：

- 普通函数的 `this` 会根据调用位置动态绑定，在 `setTimeout` 里其实是全局调用。
- 箭头函数不会创建自己的 `this`，而是“继承外部上下文的 `this`”，这里的外层是 `sayHi` 方法，`this` 就是 `person`。

这种写法在异步回调、事件监听、数组遍历中都非常实用，可以避免频繁使用 `.bind(this)` 或 `const self = this` 的老写法。

1. 原型、原型链

一、原型 (Prototype) 的概念

1.1 定义

- **原型**: 在 JavaScript 中, 每个函数 (构造函数) 都有一个属性叫做 `prototype`, 它是一个对象。通过这个对象, 构造函数创建的所有实例可以共享属性和方法。
- **对象的隐式原型**: 每个通过字面量或构造函数创建的对象, 都有一个内部属性 `[[Prototype]]` (在现代浏览器中可以通过 `__proto__` 访问), 它指向创建该对象时继承的原型对象。

1.2 如何工作

- 当我们在一个对象上访问一个属性或方法时, 首先会在该对象自身 (own property) 上查找;
- 如果没有找到, 就会沿着对象的隐式原型 (`[[Prototype]]`) 向上查找, 直到找到对应的属性或达到了原型链的末尾 (即 `null`)。

1.3 示例

```
function Person(name) {  
  this.name = name;  
}  
  
Person.prototype.sayHello = function () {  
  console.log("Hello, my name is " + this.name);  
};  
  
const alice = new Person("Alice");  
alice.sayHello(); // 输出: Hello, my name is Alice
```

在这个例子中:

- `Person.prototype` 对象提供了一个共享的方法 `sayHello`;
- 实例 `alice` 的隐式原型 (`alice.__proto__`) 指向 `Person.prototype`, 因此 `alice` 能够调用 `sayHello` 方法。

二、原型链 (Prototype Chain) 的概念

2.1 定义

- **原型链** 是由对象的 `[[Prototype]]` 属性构成的一条链, 它描述了对象属性查找的过程。
- 当访问一个对象的属性时, 如果属性不存在于对象本身, 就会从该对象的原型开始, 逐级向上查找, 直到查找到顶层对象 `Object.prototype` (它的原型为 `null`) 或没有查找到。

2.2 如何工作

- 这种通过原型层层查找的机制就是原型链, 确保了对象可以继承并复用共享的属性或方法。
- 若原型链上某一环节找到了该属性, 则直接返回该属性; 否则, 最终返回 `undefined`。

2.3 示例

```
const animal = {
  eats: true,
};

const rabbit = Object.create(animal); // rabbit 的隐式原型指向 animal
rabbit.hops = true;

console.log(rabbit.eats); // 输出 true, 属性 eats 来自原型 animal
```

在这个例子中：

- `rabbit` 对象本身只有属性 `hops`，但它的原型是 `animal`，因此能够访问 `animal` 对象中的 `eats` 属性。

三、好处

3.1 节省内存

- 通过共享原型上的方法和属性，各个实例之间无需重复存储相同的方法，实现内存高效利用。

3.2 实现继承

- 原型链机制让 JavaScript 实现了简单而灵活的继承方式，我们可以让一个对象继承另一个对象的属性和方法，而不需要复杂的类结构或多重继承体系。

3.3 动态性与灵活性

- 可以在运行时修改对象的原型，从而影响所有继承自该原型的对象，这种灵活性使得动态扩展成为可能。

3.4 方法共享

- 对象实例共享原型对象上的方法，减少了不必要的副本生成，有助于提高代码复用性与维护性。

四、应用场景

4.1 构造函数与实例共享

- **场景：**使用构造函数创建多个对象实例，希望这些实例共享某些方法或者属性。
- **实现：**将共享的方法和属性定义在构造函数的 `prototype` 对象上。

```
function Car(model) {
  this.model = model;
}

Car.prototype.start = function () {
  console.log(this.model + " 启动了!");
};

const car1 = new Car("BMW");
const car2 = new Car("Audi");
car1.start(); // BMW 启动了!
car2.start(); // Audi 启动了!
```

4.2 对象继承与属性共享

- **场景**：设计多个对象之间的继承关系，例如动物、植物等分类。
- **实现**：利用 `Object.create` 或构造函数实现继承，构建层级关系。

```
const animal = {
  walk: function () {
    console.log("动物在走路");
  },
};

const dog = Object.create(animal);
dog.bark = function () {
  console.log("汪汪");
};

dog.walk(); // 动物在走路
dog.bark(); // 汪汪
```

4.3 动态扩展对象

- **场景**：在项目运行中需要为对象动态添加公共方法或属性。
- **实现**：修改对象的原型，使所有现有和未来的实例都能获得这些新增的功能。

```
function User(name) {
  this.name = name;
}

const user1 = new User("Tom");
// 动态添加新方法到原型上
User.prototype.sayHi = function () {
  console.log("Hi, " + this.name);
};

user1.sayHi(); // Hi, Tom
```

五、总结

- **原型**：每个对象都有一个指向其“父级”对象的引用（即原型），构造函数通过 `prototype` 属性定义这些共享的特性和方法。
- **原型链**：对象通过链式结构访问继承自原型的属性和方法，形成一个查找路径。
- **优点**：节省内存、实现继承、提高代码复用、增加灵活性。
- **应用场景**：在构造函数模式下共享方法、实现对象之间的继承以及动态扩展对象功能等。

2. 正则表达式的尾缀有哪些，各有什么作用

正则表达式的尾缀（修饰符）有以下几种：

- **i**：表示忽略大小写。例如，`/abc/i` 可以匹配 "abc"、"Abc"、"aBc" 等。
- **g**：表示全局匹配。会查找字符串中所有匹配的项，而不是找到第一个就停止。例如，`/a/g` 会在 "banana" 中找到所有的 "a"。
- **m**：表示多行匹配。在处理多行文本时，`^` 和 `$` 可以匹配每一行的开头和结尾。例如，`/^abc/m` 可以匹配多行文本中每一行以 "abc" 开头的部分。

- **s**：允许 `.` 匹配合换行符。默认情况下，`.` 不能匹配合换行符，使用 `s` 修饰符后可以。例如，`/a.b/s` 可以匹配 `"a\nb"`。
- **u**：启用 Unicode 模式。可以正确处理大于 `\uFFFF` 的 Unicode 字符。例如，`/^\u{1F600}$/u` 可以匹配表情符号 😊。
- **y**：粘连匹配。从正则表达式的 `lastIndex` 属性指定的位置开始匹配，并且必须在该位置找到匹配项。例如：

```
const str = 'abcabc';
const regex = /abc/y;
regex.lastIndex = 3;
console.log(regex.test(str));
```

3. 正则全局匹配和贪婪

- **全局匹配**：使用 `g` 修饰符实现。全局匹配会在字符串中查找所有符合正则表达式的匹配项，而不是找到第一个就停止。例如：

```
const str = 'abcabc';
const regex = /abc/g;
const matches = str.match(regex);
console.log(matches); //["abc","abc"]
```

- **贪婪匹配**：正则表达式默认是贪婪匹配，也就是尽可能多地匹配符合条件的字符。例如，`/a+/` 会匹配尽可能多的连续的 `"a"`。若要实现非贪婪匹配，可在量词后面加上 `?`。例如，`/a+?/` 会匹配尽可能少的连续的 `"a"`。示例如下：

```
const str = 'aaaba';
const greedyRegex = /a+/;
const nonGreedyRegex = /a+?/;
console.log(str.match(greedyRegex)); //["aaa"]
console.log(str.match(nonGreedyRegex)); //["a"]
```

4. 前端规范的一些新特性

- **ES6+ 语法**：像箭头函数、模板字符串、解构赋值、`Promise`、`async/await` 等，这些特性让 JavaScript 代码更简洁易读。
- **CSS 新特性**：如 CSS Grid、Flexbox 布局，能够更方便地实现复杂的页面布局；`var()` 函数可以定义和使用 CSS 变量；`@media` 媒体查询能实现响应式设计。
- **HTML5 新特性**：新增了语义化标签，像 `<header>`、`<nav>`、`<article>`、`<section>`、`<footer>` 等，提高了代码的可读性和可维护性；还支持 Web Storage (`localStorage` 和 `sessionStorage`)、Web Workers 等。
- **Web Components**：包括自定义元素、Shadow DOM 和 HTML 模板，可创建可复用的组件。

5. BigInt

`BigInt` 是 JavaScript 中一种新的数字类型，用于表示任意大的整数。在传统的 JavaScript 里，`Number` 类型能表示的最大安全整数是 `Number.MAX_SAFE_INTEGER` (即 `2**53 - 1`)，超过这个范围就会出现精度丢失的问题。而 `BigInt` 可以处理任意大的整数。创建 `BigInt` 有两种方式：在数字后面加 `n` 或者使用 `BigInt()` 函数。示例如下：

```
const bigInt1 = 123456789012345678901234567890n;  
const bigInt2 = BigInt('123456789012345678901234567890');  
console.log(bigInt1 + bigInt2);
```

6. TS 内置工具类型

TypeScript 提供了一些内置的工具类型，方便开发者进行类型转换和操作。常见的有：

- `Partial<T>`：将类型 `T` 的所有属性变为可选属性。例如：

```
interface User {  
  name: string;  
  age: number;  
}  
  
type PartialUser = Partial<User>;  
const partialUser: PartialUser = { name: 'John' };
```

- `Required<T>`：将类型 `T` 的所有可选属性变为必需属性。
- `Readonly<T>`：将类型 `T` 的所有属性变为只读属性。
- `Pick<T, K>`：从类型 `T` 中选取部分属性 `K` 组成新的类型。例如：

```
interface User {  
  name: string;  
  age: number;  
  email: string;  
}  
  
type NameAndAge = Pick<User, 'name' | 'age'>;  
const nameAndAge: NameAndAge = { name: 'John', age: 30 };
```

- `Omit<T, K>`：从类型 `T` 中排除部分属性 `K` 组成新的类型。

工具类型	描述
<code>Partial<T></code>	将类型 <code>T</code> 的所有属性变为可选
<code>Required<T></code>	将类型 <code>T</code> 的所有属性变为必填
<code>Readonly<T></code>	将类型 <code>T</code> 的所有属性变为只读
<code>Record<K, T></code>	构造一个对象类型，其属性键为 <code>K</code> ，值类型为 <code>T</code>
<code>Pick<T, K></code>	从类型 <code>T</code> 中挑选一组属性组成新类型
<code>Omit<T, K></code>	从类型 <code>T</code> 中移除指定属性组成新类型
<code>Exclude<T, U></code>	从联合类型 <code>T</code> 中排除可以赋值给 <code>U</code> 的类型
<code>Extract<T, U></code>	从联合类型 <code>T</code> 中提取可以赋值给 <code>U</code> 的类型
<code>NonNullable<T></code>	移除类型中的 <code>null</code> 和 <code>undefined</code>
<code>ReturnType<T></code>	获取函数 <code>T</code> 的返回类型
<code>Parameters<T></code>	获取函数 <code>T</code> 的参数类型组成的元组
<code>ConstructorParameters<T></code>	获取构造函数类型参数组成的元组
<code>InstanceType<T></code>	获取构造函数返回的实例类型
<code>ThisParameterType<T></code>	提取函数类型 <code>T</code> 中的 <code>this</code> 类型
<code>OmitThisParameter<T></code>	移除函数类型 <code>T</code> 的 <code>this</code> 参数

7. Webpack 与 vite的配置

✓ 一、Webpack 配置示例 (`webpack.config.js`)

```
// webpack.config.js
const path = require('path');

module.exports = {
  entry: './src/index.js', // 入口文件
  output: {
    filename: 'bundle.js', // 输出文件名
    path: path.resolve(__dirname, 'dist'), // 输出路径
    clean: true, // 每次打包清理旧文件
  },
  module: {
    rules: [
      {
        test: /\.js$/, // 处理 JS
        exclude: /node_modules/,
        use: 'babel-loader', // 这里使用 `babel-loader` 处理 `.js` 文件
      },
      {
        test: /\.css$/, // 处理 CSS
        use: ['style-loader', 'css-loader'],
      },
    ],
  },
}
```

```

    ],
  },
  devServer: {
    static: './dist',
    hot: true,
    open: true,
    port: 3000,
  },
  mode: 'development', // `mode`: 指定打包模式, 有 `development` 和 `production` 两种
};

```

✓ 二、Vite 配置示例 (vite.config.js)

```

// vite.config.js
import { defineConfig } from 'vite';
import vue from '@vitejs/plugin-vue';

export default defineConfig({
  plugins: [vue()], // 使用 Vue 插件, React 项目换成 @vitejs/plugin-react
  // 本地开发服务器相关配置
  server: {
    port: 3000, // 启动本地开发服务器时使用的端口号
    open: true, // 是否自动在默认浏览器中打开项目页面
  },

  // 构建相关配置
  build: {
    outDir: 'dist', // 构建完成后文件输出的目录 (默认也是 dist, 可自定义)
  },
});

```

🔍 Webpack vs Vite 对比

维度	Webpack	Vite
启动速度	慢 (需要打包)	快 (基于原生 ES 模块, 冷启动秒开)
构建速度	慢 (全部打包)	快 (使用 Rollup 进行高效构建)
热更新	较慢 (依赖 HMR)	快速响应变化
配置复杂度	较高 (需写 loader、plugin)	简单 (开箱即用)
插件系统	成熟、强大	也支持插件, 但生态稍弱
适合项目	大型复杂项目、老项目	现代前端项目、轻量项目、组件库开发

📦 推荐使用场景

- **Webpack**: 更适合大型项目或需要复杂自定义构建流程的项目。
- **Vite**: 更适合中小型项目、快速开发、Vue/React 项目, 体验极佳。

8. 与 webpack 相关的打包工具、webpack 是如何打包的

- 相关打包工具:

- **Rollup**: 专注于 JavaScript 库的打包, 能生成更简洁的代码, 适合构建 ES 模块。
- **Parcel**: 零配置的打包工具, 使用简单, 能自动处理各种文件类型。
- **Vite**: 基于 ES 模块的快速打包工具, 开发环境启动速度快, 适合现代前端项目。
- Webpack 打包原理:
 1. **解析入口文件**: Webpack 从入口文件开始, 分析文件中的依赖关系。
 2. **构建模块图**: 通过递归分析所有依赖的模块, 构建出一个模块图。
 3. **加载模块**: 根据模块的类型, 使用相应的 loader 进行处理。
 4. **打包输出**: 将处理后的模块打包成一个或多个文件, 输出到指定的目录。

9. Babel 有什么用, 在 webpack 中有什么用

- **Babel 的作用**: Babel 是一个 JavaScript 编译器, 主要用于将 ES6+ 语法的代码转换为向后兼容的 JavaScript 代码, 以便在旧版本的浏览器或环境中运行。它可以处理箭头函数、模板字符串、`async/await` 等新特性。
- **在 Webpack 中的作用**: 在 Webpack 里, Babel 通常作为一个 loader (即 `babel-loader`) 使用。Webpack 在处理 `.js` 文件时, 会调用 `babel-loader`, 将代码交给 Babel 进行转换, 然后再进行打包。这样可以确保打包后的代码能在更多环境中运行。

10. Less 中有哪些变量

Less 是一种 CSS 预处理器, 它扩展了 CSS 的功能, 允许使用变量、嵌套规则、混合等特性。Less 中的变量使用 `@` 符号定义。示例如下:

```
// 定义变量
@primary-color: #007bff;
@font-size-base: 16px;

// 使用变量
body {
  color: @primary-color;
  font-size: @font-size-base;
}

// 变量也可以在混合中使用
.border-radius(@radius: 5px) {
  border-radius: @radius;
}

.button {
  .border-radius();
}
```

在这个例子中, `@primary-color` 和 `@font-size-base` 是定义的变量, `@radius` 是混合中的参数变量。这些变量可以在 Less 文件中多次使用, 提高了代码的可维护性和复用性。

腾讯6

Q1. 拷打项目

Q2. `created` 和 `mounted` 的区别?

在 Vue.js 中, `created` 和 `mounted` 都是生命周期钩子函数, 但它们的执行时机不同:

- `created`：在 Vue 实例创建后，数据观测和事件配置等操作完成时调用。此时，DOM 还没有被渲染，因此无法访问到 DOM 元素。适用于需要在组件加载前进行初始化的操作。
- `mounted`：在 Vue 实例挂载到 DOM 上之后调用，即所有的 DOM 渲染完成后执行。可以安全地访问和操作 DOM 元素，适用于与 DOM 交互的操作，如获取元素的大小、位置等。

简言之，`created` 是在数据和事件设置后执行的，而 `mounted` 是在组件完全挂载到页面上时执行的。

Q3. 原生 JS 如何实现发布订阅？如何通信？

原生 JavaScript 实现发布-订阅模式可以通过自定义事件管理来实现。基本的做法是创建一个事件管理器，订阅者可以订阅事件，发布者则可以发布事件通知所

有订阅者。

以下是一个简单的实现：

```
function PubSub() {
  this.events = {};
}

// 订阅方法
PubSub.prototype.subscribe = function(event, callback) {
  if (!this.events[event]) {
    this.events[event] = [];
  }
  this.events[event].push(callback);
};

// 发布方法
PubSub.prototype.publish = function(event, data) {
  if (this.events[event]) {
    this.events[event].forEach(callback => callback(data));
  }
};

// 示例
const pubSub = new PubSub();

pubSub.subscribe('message', function(data) {
  console.log('Received message:', data);
});

pubSub.publish('message', 'Hello, world!');
```

在这个例子中，`subscribe` 用于订阅事件，`publish` 用于发布事件。通过这种方式，可以实现对象间的解耦通信。

Q4. 事件捕获和冒泡的过程

事件流分为三个阶段：捕获阶段、目标阶段和冒泡阶段。

1. 捕获阶段 (Capture Phase)：

- 事件从根节点开始向目标元素传播，即事件首先会在外层元素上触发（父元素到子元素），这是捕获阶段。
- 如果事件监听器设置为捕获模式，它会在捕获阶段触发。

2. 目标阶段 (Target Phase)：

- 当事件到达目标元素时，会触发该元素的事件监听器。

3. 冒泡阶段 (Bubble Phase) :

- 事件从目标元素开始向外传播，直到根节点（子元素到父元素）。这是冒泡阶段。
- 如果事件监听器设置为冒泡模式，它会在冒泡阶段触发。

默认情况下，浏览器会执行冒泡阶段，但也可以通过 `event.stopPropagation()` 来阻止事件的传播，或通过设置事件监听器的 `capture` 选项来开启捕获阶段。

例如，假设有以下 HTML 结构：

```
<div id="parent">
  <button id="child">Click me</button>
</div>
```

监听事件的代码可以如下：

```
document.getElementById('parent').addEventListener('click', function() {
  console.log('Parent clicked');
}, false); // false 表示冒泡阶段

document.getElementById('child').addEventListener('click', function() {
  console.log('Child clicked');
}, false); // false 表示冒泡阶段
```

如果点击 `button`，事件会先到达 `child`，然后冒泡到 `parent`，并输出：

```
Child clicked
Parent clicked
```

如果将 `parent` 的事件监听器设置为捕获阶段：

```
document.getElementById('parent').addEventListener('click', function() {
  console.log('Parent clicked');
}, true); // true 表示捕获阶段
```

则事件会首先触发 `parent`，再触发 `child`，并输出：

```
Parent clicked
Child clicked
```

Q5. CSS的 `var` 是什么？定义如何？

CSS中的 `var()` 是用来定义和使用**CSS变量**的功能，它使得CSS更加灵活，支持在多个地方引用相同的值。CSS变量是可以在CSS文件中动态赋值并复用的属性。

定义和使用CSS变量：

1. **定义变量：**使用 `--` 前缀来定义变量。通常定义在 `:root` 选择器中，这样就可以在整个文档中使用这些变量。

```
:root {
  --primary-color: #3498db;
  --secondary-color: #2ecc71;
  --font-size: 16px;
}
```

2. **使用变量**：使用 `var()` 函数来调用变量的值。

```
body {
  color: var(--primary-color);
  font-size: var(--font-size);
}

h1 {
  color: var(--secondary-color);
}
```

解释：

- `--primary-color`, `--font-size` 等都是自定义的CSS变量。
- 通过 `var(--primary-color)` 可以引用这些变量的值。
- CSS变量的作用域是由它们的定义位置决定的，可以限定在某个元素、类或者全局生效。

优势：

- **可复用性**：避免在多个地方重复相同的值，增强代码的维护性。
- **动态更新**：通过JavaScript可以动态修改CSS变量的值。

Q6. HTTP和HTTPS的区别？

HTTP (HyperText Transfer Protocol) :

- **无加密**：数据以明文形式传输，易受中间人攻击。
- **端口**：默认使用端口80。
- **性能**：由于没有加密层，HTTP在处理时相对更轻便。
- **适用场景**：适用于对安全性要求不高的场景，如公开的网页。

HTTPS (HyperText Transfer Protocol Secure) :

- **加密传输**：使用SSL/TLS协议对数据进行加密，确保数据在传输过程中不会被窃听或篡改。
- **端口**：默认使用端口443。
- **安全性**：支持身份验证，防止中间人攻击，确保客户端与服务器之间的通信安全。
- **性能**：由于加密解密的开销，HTTPS相比HTTP略慢，但随着技术进步，性能差异已经较小。
- **适用场景**：适用于涉及敏感信息（如登录、支付等）的场景。

区别：

1. **数据加密**：HTTPS对数据进行加密，HTTP不加密。
2. **认证功能**：HTTPS提供服务器身份认证，HTTP没有身份认证功能。
3. **性能**：HTTPS需要更多的计算资源来加密和解密数据，HTTP无此开销。
4. **安全性**：HTTPS比HTTP安全，防止数据被窃取或篡改。

Q7. 在前端向后端传递参数时，如何通知后端body里面的数据类型？

在前端发送请求时，通常通过HTTP请求的 `Content-Type` 头来告知后端数据的格式。常见的 `Content-Type` 类型包括：

1. `application/json`：表示请求体是JSON格式的数据，适用于RESTful API。

示例：

```
fetch('/api', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json'
  },
  body: JSON.stringify({ key: 'value' })
});
```

2. `application/x-www-form-urlencoded`：表示请求体是表单数据，通常用于传统表单提交。

示例：

```
fetch('/api', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/x-www-form-urlencoded'
  },
  body: 'key=value&anotherKey=anotherValue'
});
```

3. `multipart/form-data`：表示请求体用于文件上传。

示例：

```
let formData = new FormData();
formData.append('file', fileInput.files[0]);
fetch('/upload', {
  method: 'POST',
  body: formData
});
```

4. `text/plain`：表示请求体是纯文本。

Q8. 后端如何告诉前端返回值的类型？

后端通过HTTP响应头中的 `Content-Type` 字段来告知前端返回的数据类型。常见的返回类型有：

1. `application/json`：表示返回JSON格式的数据。

示例（Node.js后端）：

```
res.setHeader('Content-Type', 'application/json');
res.json({ message: 'Hello' });
```

2. `text/html`：表示返回HTML文档。

示例：

```
res.setHeader('Content-Type', 'text/html');
res.send('<h1>Hello world</h1>');
```

3. `application/xml`: 表示返回XML格式的数据。

示例:

```
res.setHeader('Content-Type', 'application/xml');  
res.send('<message>Hello world</message>');
```

4. `text/plain`: 表示返回纯文本数据。

Q9. 时间复杂度和空间复杂度的意思?

1. **时间复杂度**: 表示算法执行所需时间随输入规模变化的增长趋势。通常用大O符号 ($O(n)$) 表示。时间复杂度的计算反映了算法效率如何随输入规模的增大而变化。
 - $O(1)$: 常数时间复杂度, 表示执行时间不受输入规模影响。
 - $O(n)$: 线性时间复杂度, 表示执行时间与输入规模成正比。
 - $O(n^2)$: 平方时间复杂度, 表示执行时间与输入规模的平方成正比。
2. **空间复杂度**: 表示算法执行过程中所需的额外空间随输入规模的增长而增长的趋势。也是用大O符号表示。
 - $O(1)$: 常数空间复杂度, 表示算法占用的空间不随输入规模变化。
 - $O(n)$: 线性空间复杂度, 表示算法占用的空间与输入规模成正比。

Q10. WebSocket和MQTT的区别?

1. **WebSocket**:
 - 双向通信协议。
 - 基于TCP协议, 适用于实时数据交换。
 - 可以通过浏览器与服务器建立持久连接, 实现实时通信。
 - 用于聊天应用、实时通知等。
2. **MQTT**:
 - 发布/订阅模式, 适用于IoT (物联网) 设备。
 - 基于TCP协议, 可以通过较低带宽和不稳定网络环境下保持可靠的消息传递。
 - 轻量级, 适用于低带宽、高延迟的环境。
 - 用于设备间通信、智能家居等。

区别:

- **通信模式**: WebSocket是点对点的, 而MQTT是发布/订阅模式。
- **适用场景**: WebSocket适用于实时网页应用, MQTT适用于物联网设备。
- **协议开销**: MQTT相对轻量, 更适合低带宽环境。

Q11. 动态规划算法和贪心算法的区别?

1. **动态规划**:
 - 解决问题时, 分解成子问题并通过保存子问题的解来避免重复计算。
 - 通常使用递归和记忆化技术来实现。
 - 适用于最优解问题, 如最短路径、最大子数组和背包问题。
 - 时间复杂度通常较高, 但可以通过保存中间结果来提高效率。
2. **贪心算法**:
 - 每一步选择局部最优解, 最终通过局部最优解得到全局最优解。
 - 不考虑全局最优解的可能性, 贪心选择会在每个阶段做出局部最优的决策。
 - 通常应用于特定类型的问题, 如活动选择、哈夫曼编码等。

区别：

- **决策方式**：动态规划考虑全局最优解，贪心算法只考虑局部最优解。
- **问题类型**：动态规划适用于最优解问题，贪心算法适用于满足“贪心选择性质”的问题。

Q12. 数据库中，向量和索引的关系是什么？

1. **向量**：向量是用来表示数据的一种方式，通常用于表示高维数据或复杂数据（如文本、图像等）。在数据库中，向量可以是一个多维数组，表示某种特征。
2. **索引**：索引是用来加速数据库查询的一种数据结构。对于高维数据，如文本的向量表示，传统的索引方法（如B树）效率低下，因此需要使用专门的向量索引结构（如LSH、KD树）来提高查询效率。

关系：

- 向量索引用于高效查找相似的数据。向量索引通过将数据映射到特征空间，支持高效的近似查询，尤其在大数据量的场景下。

Q13. SQL和NoSQL数据库的区别？

1. **SQL数据库**（如MySQL、PostgreSQL）：
 - 基于关系模型，数据存储在表格中，数据之间存在明确的关系。
 - 使用结构化查询语言（SQL）来操作数据。
 - 支持ACID（原子性、一致性、隔离性、持久性）事务。
 - 适用于数据结构固定且关系复杂的应用。
2. **NoSQL数据库**（如MongoDB、Cassandra）：
 - 非关系型数据库，数据可以是键值对、文档、列族或图形等形式。
 - 不使用SQL语言，通常提供简单的查询接口。
 - 适合处理海量的、非结构化或半结构化的数据。
 - 高度可扩展，适用于分布式系统和快速开发。

区别：

- **结构**：SQL使用表格存储，NoSQL使用灵活的数据结构。
- **扩展性**：NoSQL更适合大规模的水平扩展。
- **事务支持**：SQL支持ACID事务，NoSQL通常不严格支持。

Q14. 多个WebWorker之间如何通信？

WebWorkers之间的通信通常是通过 `postMessage()` 进行的，它允许WebWorker之间传递消息。每个Worker都可以监听 `message` 事件来接收消息。

```
// worker1.js
onmessage = function (e) {
  postMessage('Message from worker1: ' + e.data);
}

// worker2.js
onmessage = function (e) {
  postMessage('Message from worker2: ' + e.data);
}
```

多个Worker可以互相传递数据，但它们不能直接共享内存。可以通过 `postMessage()` 方法传递消息，并通过 `onmessage` 事件接收。

Q15. WebWorker和JS页面是值传递还是引用传递？

Web Worker 与主线程（即 JS 页面）之间的数据传递方式是**值传递（结构化克隆）**，而不是引用传递。每次通过 `postMessage()` 传递数据时，数据会被克隆（深拷贝）。因此，WebWorker和JS页面之间的变量是独立的，不会相互影响。

- Web Worker 与主线程之间通过 `postMessage()` 进行通信。
- 实际上使用的是一种叫做 **结构化克隆算法（Structured Clone Algorithm）** 的机制。
- 即：**主线程将数据的“深拷贝”传递给 Worker，反之亦然。**
- 所以不是共享引用，也不能直接修改彼此的数据。

主线程代码（index.html）：

```
<script>
  const worker = new Worker('worker.js');

  const obj = { message: 'hello', arr: [1, 2, 3] };

  worker.postMessage(obj); // 传递对象

  // 修改原对象
  obj.message = 'changed';
  obj.arr.push(4);

  worker.onmessage = (e) => {
    console.log('主线程收到: ', e.data);
  };
</script>
```

Worker 文件（worker.js）：

```
onmessage = function (e) {
  console.log('Worker收到的对象: ', e.data);
  // 修改数据
  e.data.message = 'modified in worker';
  postMessage(e.data);
};
```

输出结果（在控制台）：

```
worker收到的对象: { message: 'hello', arr: [1, 2, 3] }
主线程收到: { message: 'modified in worker', arr: [1, 2, 3] }
```

说明：

- 虽然主线程 later 修改了 `obj.message` 和数组，但 Worker 中的副本并没有变化。
- 说明数据是被“复制”而非“引用传递”。
- Worker 修改的副本也不会影响主线程原对象。

✓ 总结

传递方式	类型	是否共享引用
默认（结构化克隆）	一般对象、数组、字符串等	✗ 否
Transferable	ArrayBuffer、MessagePort 等	✓ 共享所有权（但只一方可用）

Q16. BOM和DOM的区别？

- 1. **BOM（浏览器对象模型）**：用于表示浏览器窗口和与浏览器交互的对象。通过BOM，JavaScript可以操作浏览器的窗口、历史记录、导航、定位等。
- 2. **DOM（文档对象模型）**：用于表示网页结构及其内容。通过DOM，JavaScript可以动态修改网页的结构、内容、样式等。

✓ 一图看懂：BOM vs DOM 结构图



✓ 二、核心区别对比表

比较维度	BOM (Browser Object Model)	DOM (Document Object Model)
中文名称	浏览器对象模型	文档对象模型
操作对象	浏览器窗口、历史、地址栏等	网页内容结构 (HTML/XML)
核心对象	<code>window</code>	<code>document</code>
主要用途	控制浏览器行为	操作网页内容结构
代表	<code>window</code> <code>location</code> <code>history</code> <code>navigator</code> <code>screen</code>	<code>document.getElementById()</code>

对比维度标准	window、location、history、navigator、screen	节点增删改查
	BOM (Browser Object Model)	DOM (Document Object Model)
	非W3C标准, 各浏览器实现可能不同	W3C 标准, 跨浏览器一致性好

三、简单举例说明

📌 BOM 示例:

```
// 弹出对话框
window.alert("Hello");

// 跳转页面
location.href = "https://example.com";

// 获取浏览器信息
console.log(navigator.userAgent);

// 后退一页
history.back();
```

📌 DOM 示例:

```
// 获取元素
const title = document.getElementById("title");

// 修改文本
title.innerText = "新标题";

// 创建并添加元素
const p = document.createElement("p");
p.textContent = "Hello DOM!";
document.body.appendChild(p);
```

四、总结一句话

DOM 操作网页结构内容, BOM 操作浏览器窗口行为。BOM 是大环境, DOM 是具体页面内容。

Q17. SSE、WebSocket和HTTP三者之间的区别?

一、三者简述

名称	简称	说明
HTTP	-	单向通信, 客户端请求、服务端响应
WebSocket	WS	双向通信, 全双工、实时性强
Server-Sent Events	SSE	服务端向客户端单向推送, 基于 HTTP 长连接

✓ 二、详细对比表格

特性/协议	HTTP	WebSocket	SSE (Server-Sent Events)
连接方式	请求-响应 (短连接)	持久连接 (ws:// / wss://)	持久连接 (http)
通信方向	客户端 → 服务端	双向 (客户端 ↔ 服务端)	单向 (服务端 → 客户端)
实时性	差 (需轮询)	优 (事件驱动)	较好 (自动重连)
复杂度	最简单	较复杂 (需要协议升级)	较简单 (浏览器原生支持)
兼容性	所有浏览器	IE10+ 现代浏览器支持	IE 不支持 (Edge、Chrome 支持)
自动重连	✗	需手动实现	✓ 自动重连
支持跨域	✓ (需 CORS)	✓ (需 CORS)	✓ (需 CORS)
数据格式	任意 (JSON、XML 等)	任意 (二进制、JSON等)	纯文本 (推荐 UTF-8)
适合场景	常规请求 (REST API)	聊天、游戏、实时协作、股票等	实时通知、新闻、系统告警等

✓ 三、应用场景对比图解：

应用场景	推荐方案
新闻通知流	SSE ✓
聊天室/在线游戏	WebSocket ✓
用户登录请求	HTTP ✓
股票/币价推送	WebSocket ✓
后台系统日志推送	SSE ✓

✓ 四、示意图对比

HTTP：

[Client] --(Request)--> [Server]
[Client] <--(Response)-- [Server]

WebSocket:

```
[Client] <====双向持久连接====> [Server]
```

SSE:

```
[Client] --(发起连接)--> [Server]
[Client] <====持续推送数据==== [Server]
```

✓ 五、总结一句话:

- **HTTP**: 请求响应型, 最常用。
- **WebSocket**: 实时双向通信, 适合高交互性应用。
- **SSE**: 服务端推送, 轻量简单, 适合通知流、事件流。

Q18: 怎么进行两个页面通信?

一、同源页面之间的通信方式

1 `window.open` (新窗口与打开它的窗口通信)

- **适用场景**: 页面 A 使用 `window.open()` 打开了页面 B。
- **通信方式**: A 可以访问 B 的 `window`, B 可以通过 `window.opener` 访问 A。

```
// 页面A
const win = window.open('pageB.html');
win.postMessage('Hello from A', '*');

// 页面B
window.addEventListener('message', (e) => {
  console.log('收到来自A的消息:', e.data);
  e.source.postMessage('收到啦, B在回信', e.origin);
});
```

2 `localStorage` + `storage` 事件

- **适用场景**: 同源但不同窗口/标签页之间通信 (无法直接访问 `window`)
- **机制**: 对 `localStorage` 的改动会在其他标签页触发 `storage` 事件。

```
// 页面A
localStorage.setItem('message', 'Hello B');

// 页面B
window.addEventListener('storage', (e) => {
  if (e.key === 'message') {
    console.log('收到A的消息:', e.newValue);
  }
});
```

3 BroadcastChannel API (推荐)

- **适用场景**: 同源页面间广播通信, 不管是否打开多个标签页或 iframe。
- **优点**: 简单、高效、事件驱动, 不依赖于 DOM 或窗口结构。

```
const channel = new BroadcastChannel('my_channel');

// 页面A 或 B
channel.postMessage('Hello from one tab');

// 页面A 或 B
channel.onmessage = (e) => {
  console.log('收到广播:', e.data);
};
```

4 SharedWorker (高级场景)

- **适用场景**: 多个页面共享同一个后台线程, 同源。
- **优点**: 支持双向通信, 适合共享数据或 WebSocket。

```
// shared-worker.js
onconnect = function (e) {
  const port = e.ports[0];
  port.onmessage = function (event) {
    console.log('worker 收到: ', event.data);
    port.postMessage('返回消息: ' + event.data);
  };
};

// 页面
const worker = new SharedWorker('shared-worker.js');
worker.port.start();
worker.port.postMessage('hello');
worker.port.onmessage = (e) => {
  console.log('页面收到worker的回应:', e.data);
};
```

🌐 二、跨域页面之间的通信方式

1 postMessage (核心方式)

- **适用场景**: 跨域或同域的 iframe、window、popup 页面之间。
- **安全性好**: 可设置 targetOrigin, 防止非法消息。

```
// 页面A 向 iframe B 发消息
iframeEl.contentWindow.postMessage('hello B', 'https://example.com');

// 页面B 接收
window.addEventListener('message', function(e) {
  console.log('接收到A发来的信息:', e.data);
});
```

2 window.name 中转法（早期跨域方式）

- 利用 `window.name` 在跨域页面跳转时不会被清除的特点实现跨域通信。
- 现代项目中较少使用，已被 `postMessage` 取代。

三、iframe 中的父子页面通信

页面关系	使用方式
父向子	<code>iframe.contentWindow.postMessage(...)</code>
子向父	<code>window.parent.postMessage(...)</code>
同域还可直接访问 DOM	如 <code>window.parent.document</code> （仅同源）

✓ 总结表格

技术	是否支持跨域	支持方向	特点
<code>postMessage</code>	✓	双向	跨域推荐，最通用
<code>localStorage + storage</code>	✗（同源）	单向（触发）	标签页间通信
<code>BroadcastChannel</code>	✗（同源）	广播	多标签页/iframe 广播
<code>window.opener</code>	✗（同源）	双向	popup 页面通信
<code>SharedWorker</code>	✗（同源）	双向	后台线程、性能好
<code>window.name</code>	✓	单向	已过时，不推荐

字节1

Q1. 手写并发调度控制器

并发调度控制器常用于限制任务的并发执行数量，防止任务过多导致资源过载。可以使用 `Promise` 和 `async/await` 来实现。

```
class ConcurrencyController {
  constructor(limit) {
    this.limit = limit; // 限制并发的数量
    this.current = 0;    // 当前并发任务数量
    this.queue = [];     // 队列，存放待执行的任务
  }

  async addTask(task) {
    if (this.current >= this.limit) {
      await new Promise(resolve => this.queue.push(resolve)); // 当并发达到限制时，
      等待
    }
    this.current++;
    try {
      await task();
    } finally {
      this.current--;
    }
  }
}
```

```

        if (this.queue.length > 0) {
            this.queue.shift(); // 执行等待队列中的任务
        }
    }
}
}

const controller = new ConcurrencyController(2); // 限制并发数量为 2
controller.addTask(async () => { /* some task */ });
controller.addTask(async () => { /* some task */ });

```

Q2. 原型链和类介绍与区别

原型链：

- JavaScript 中，每个对象都有一个内部属性 `[[Prototype]]`（可以通过 `__proto__` 访问）。
- 通过原型链，子对象可以访问父对象的属性和方法。

类 (Class)：

- ES6 引入的语法糖，用于更简单地创建构造函数和继承。
- 通过类可以清晰地定义方法和继承关系。

区别：

- **继承方式：**原型链通过构造函数的 `prototype` 进行继承，而类通过 `extends` 关键字进行继承。
- **语法：**类的语法更加简洁，容易理解。

```

// 原型链
function Person(name) {
    this.name = name;
}
Person.prototype.sayHello = function() {
    console.log(`Hello, ${this.name}`);
};

// 类
class Person {
    constructor(name) {
        this.name = name;
    }
    sayHello() {
        console.log(`Hello, ${this.name}`);
    }
}

```

Q3. 箭头函数

箭头函数是 ES6 新增的函数表达式，具有简洁的语法和不同的 `this` 绑定行为。

- 语法：`(param1, param2) => { return param1 + param2; }`
- 特点：
 - 不绑定 `this`，`this` 继承自外层作用域。
 - 不可以作为构造函数。

- 无 `arguments` 对象。

```
const add = (a, b) => a + b;
```

Q4. TS、装饰器介绍

TypeScript:

- TypeScript 是 JavaScript 的超集，增加了静态类型检查、接口、泛型等功能。
- TS 的主要优势是提供静态类型检查，能帮助开发者减少错误。

装饰器 (Decorator) :

- 装饰器是 ES7 提案中的一部分，允许通过特殊的标注语法来修改类的行为。
- 用途：可以用于类、方法、属性和方法参数。

```
function validate(target: any, propertyKey: string, descriptor:
PropertyDescriptor) {
  const originalMethod = descriptor.value;
  descriptor.value = function (...args: any[]) {
    if (args.some(arg => typeof arg !== 'number')) {
      throw new Error('All arguments must be numbers');
    }
    return originalMethod.apply(this, args);
  };
}

class Calculator {
  @validate
  add(a: number, b: number): number {
    return a + b;
  }
}

const calc = new Calculator();
calc.add(3, 4); // 正常工作
calc.add(3, '4'); // 抛出错误
```

✓ 使用 `@validate` 的结果:

```
const calc = new Calculator();
calc.add(3, 4); // ✓ 正常工作，结果为 7
calc.add(3, '4'); // ✗ 抛出错误: All arguments must be numbers
```

- 在调用 `add(3, '4')` 时，装饰器会先检查参数类型。
- `typeof '4' === 'string'`，不满足数字类型，抛出异常。
- 所以此时不会执行 `add` 方法本身，直接中断抛错，代码更安全、可控。

✗ 不使用 `@validate` 的结果:

```
const calc = new Calculator();
calc.add(3, 4);      // ✔️ 正常工作, 结果为 7
calc.add(3, '4');    // ❌ 执行结果为 '34'
```

- JS 是弱类型语言, `3 + '4'` 会触发类型自动转换, 结果为字符串 `'34'`。
- 不报错, 但结果是 **逻辑错误**, 可能造成后续 Bug, 且不容易察觉。
- 这就是**静默错误**的问题: 代码看起来运行正常, 但实际数据是错的。

✔️ 装饰器带来的好处:

项目	使用装饰器 (@validate)	不使用装饰器
类型检查	自动检查, 抛出明确错误	无检查, 易出错
安全性	高, 避免意外类型操作	低, 容易产生隐性 bug
可读性/可维护性	明确声明方法校验规则	校验逻辑必须手动加在方法里
可复用性	高 (一个装饰器多个方法可用)	低 (每个方法重复写)

Q5. 手写深拷贝, 增加循环引用、depth 和特殊类型处理

```
function deepClone(obj, depth = Infinity, map = new WeakMap()) {
  if (depth === 0) return obj;

  if (obj === null || typeof obj !== 'object') return obj;

  if (map.has(obj)) return map.get(obj); // 处理循环引用
  const clone = Array.isArray(obj) ? [] : {};

  map.set(obj, clone);

  for (let key in obj) {
    if (obj.hasOwnProperty(key)) {
      clone[key] = deepClone(obj[key], depth - 1, map); // 递归克隆
    }
  }

  return clone;
}
```

Q6. WeakMap、Map、Object 介绍与区别

- `WeakMap`:
 - 键必须是对象, 值可以是任何类型。
 - 不会阻止垃圾回收, 键被销毁后, `WeakMap` 中的键值对会自动清除。

- **Map**：
 - 键值对存储，可以是任何类型的键。
 - 会保持键值对的插入顺序。
- **Object**：
 - 传统的 JavaScript 对象，键只能是字符串或符号。

区别：

- **WeakMap** 适用于不希望阻止对象被垃圾回收的场景。
- **Map** 和 **Object** 适用于需要保持键值对的顺序和支持多种键类型的场景。

Q7. Vue 源码中使用 WeakMap 和 Set 的场景介绍

- **WeakMap**：用于存储和管理依赖关系的缓存，防止因为 Vue 的响应式系统导致内存泄漏。
- **Set**：用于管理某个对象的依赖集合，Vue 会通过 **Set** 来管理所有依赖该对象的视图。

Q8. Composition API 和 Options API 介绍与区别

Options API

- 传统的 Vue 组件定义方式，使用 **data**、**methods**、**computed** 等选项来定义组件。

Composition API

- Vue 3 引入的新 API，允许通过 **setup()** 函数组织逻辑，支持更灵活的代码复用和组织。

区别

- **灵活性**：Composition API 更灵活，可以复用功能和逻辑。
- **可维护性**：对于复杂的组件，Composition API 更具可维护性。

Q9. 浏览器缓存，部署时文件具体的缓存类型

- **浏览器缓存**：
 - **强缓存**：通过 **Cache-Control** 或 **Expires** 设置，避免每次请求都去服务器获取资源。
 - **协商缓存**：通过 **ETag** 和 **Last-Modified** 配合 **If-None-Match** 或 **If-Modified-Since** 来判断文件是否变化。
- **部署缓存策略**：
 - 版本化文件名，利用浏览器缓存。
 - 使用 **service worker** 缓存部分资源。
 - **Service Worker** 是浏览器运行在主线之外的**独立线程**，可以拦截和处理网络请求，实现离线缓存、消息推送等功能。

Q10. HTTPS、RSA 和 AES 加密介绍，前端加密和 HTTPS 加密区别，前端加密具体实现思路与场景

- **HTTPS**：利用 SSL/TLS 协议对 HTTP 流量进行加密。
- **RSA**：非对称加密算法，常用于加密密钥交换，确保数据传输安全。
- **AES**：对称加密算法，用于加密大量数据。

前端加密与 HTTPS 加密区别：

- **HTTPS 加密**：主要用于数据传输过程中的加密，防止中间人攻击。
- **前端加密**：可用于本地加密数据（如密码、用户信息），加密后的数据需要安全存储。

前端加密的实现思路：

- **前端加密思路**：可使用 `RSA` 进行加密，使用公钥加密后传输给服务器，服务器使用私钥解密。
- **场景**：比如敏感信息如密码或支付信息需要加密后传输。

Q11: Vue 3 编译流程（简化版）

Vue 3 中模板编译大致流程如下：

```
template => parse => transform => generate => render function
```

也就是：

1. `parse`：将模板字符串转为 AST（抽象语法树）

- 把 `<div>{{ msg }}</div>` 等 HTML/指令语法转为 JS 可识别的数据结构
- Vue 自带编译器使用了自己实现的 HTML parser

2. `transform`：遍历 AST，做优化和标记

- 静态提升（将不变部分提前生成）
- 插值表达式转为 `createTextVNode`
- `v-if` / `v-for` 特殊结构转换
- 标记动态节点（用于后续靶向 diff）

3. `codegen`：生成可执行的 `render` 函数

- 类似这样：

```
return function render(ctx, cache) {  
  return openBlock(), createElementBlock("div", null,  
    toDisplayString(ctx.msg))  
}
```

Vue 3 编译后 AST 简单示例

假设你有模板：

```
<div><p>{{ msg }}</p><span>静态文本</span></div>
```

`parse` 后的 AST 会是一个这样的结构（简化表示）：

```
{  
  type: 'root',  
  children: [  
    {  
      type: 'element',  
      tag: 'div',
```

```

    children: [
      {
        type: 'element',
        tag: 'p',
        children: [{ type: 'interpolation', content: 'msg' }]
      },
      {
        type: 'element',
        tag: 'span',
        children: [{ type: 'text', content: '静态文本' }]
      }
    ]
  }
]
}

```

编译优化（如静态提升）

Vue 会识别哪些节点是静态的、不会因响应式变化而更新，从而提升性能：

- `markStatic`：标记静态节点
- `hoistStatic`：将静态节点提前用变量存储

这样，静态内容只会生成一次，不会在每次更新时重新 patch。

Vue 3 的“靶向 diff”是怎么处理 DOM 结构变化的？

传统的虚拟 DOM diff 是全量对比，而 Vue 3 做了很多针对优化（如 PatchFlags）：

PatchFlags：编译时为每个节点添加动态标记

比如：

- `TEXT` 表示节点文本可能会变（如 `{{ msg }}`）
- `CLASS` 表示 class 是动态的
- `PROPS` 表示 props 中有响应式绑定
- `KEYED_FRAGMENT` 表示是有 key 的列表节点

在 patch 阶段，Vue 就只对有这些标记的部分做最小更新。

```
createVNode("div", { class: dynamicClass }, null, PatchFlags.CLASS)
```

这就叫做**靶向 diff**：精确知道哪些地方要更新，而不是整个树重新比对。

总结一句话

Vue 3 编译器通过 `parse → transform → generate` 将模板转为 render 函数，期间做了静态提升、PatchFlags 靶向标记，最终提高 diff 性能和运行效率。

Q12：讲讲websocket和它的生命周期

🌐 一、什么是 WebSocket?

WebSocket 是一种在客户端和服务器之间建立**长连接**的协议，允许**双向通信**，相比 HTTP 的“请求-响应”模式更适合实时应用，比如：

- 在线聊天、游戏
- 股票行情推送
- 实时协作编辑（如Google Docs）

🔄 二、WebSocket 生命周期详解

WebSocket 的生命周期分为 **五个阶段**：

1. CONNECTING (0) 连接中

- 客户端使用 `new WebSocket(url)` 发起连接
- WebSocket 与服务器协商协议，状态为 **CONNECTING**

```
const socket = new WebSocket("wss://example.com/socket");
```

2. OPEN (1) 连接已建立

- 握手成功，连接建立
- 客户端可以通过 `socket.send()` 发送消息

```
socket.onopen = () => {  
  console.log("连接已打开");  
  socket.send("Hello Server!");  
};
```

3. MESSAGE (实时触发)

- 服务器发送消息，触发 `onmessage` 事件
- 可持续进行消息的收发

```
socket.onmessage = (event) => {  
  console.log("接收到消息:", event.data);  
};
```

4. CLOSING (2) 关闭中

- `socket.close()` 被调用时，连接进入关闭中状态
- 此时不能发送消息，但连接还没完全断开

5. CLOSED (3) 已关闭

- 连接完全断开，触发 `onclose`
- 需要重新连接需重新执行 `new WebSocket()`

```
socket.onclose = () => {  
  console.log("连接已关闭");  
};
```

6. ERROR (错误)

- 网络问题、协议错误时触发 `onerror`
- 注意：不会给出详细错误信息（出于安全考虑）

```
socket.onerror = (error) => {  
  console.error("连接发生错误", error);  
};
```

总结生命周期状态码：

状态码	状态名称	说明
0	CONNECTING	正在连接
1	OPEN	已连接，可通信
2	CLOSING	关闭中
3	CLOSED	已关闭，连接断开

京东1

Q1：CSS 中哪些属性可以继承？

可继承属性主要集中在文本、表格、列表相关，例如：

- 文本类： `color`、 `font`、 `letter-spacing`、 `line-height`、 `word-spacing`、 `text-align`、 `visibility`、 `white-space`
- 列表类： `list-style`、 `list-style-type`、 `list-style-position`
- 表格类： `border-collapse`、 `caption-side`

可使用 `inherit` 强制继承任何属性。

Q2：CSS 选择器及其权重规则？ 如何调整？

常见选择器：

类型	示例
标签选择器	<code>div</code> 、 <code>p</code>
类选择器	<code>.box</code>
ID选择器	<code>#main</code>
属性选择器	<code>[type="text"]</code>
伪类选择器	<code>a:hover</code> 、 <code>:nth-child</code>
伪元素选择器	<code>::before</code> 、 <code>::after</code>
组合选择器	<code>div p</code> 、 <code>ul > li</code>
群组选择器	<code>h1</code> 、 <code>h2</code>

选择器权重（由小到大）：

- 通配符（*）、组合符（+、>）→ 0,0,0,0
- 标签选择器 → 0,0,0,1
- 类、伪类、属性选择器 → 0,0,1,0
- ID选择器 → 0,1,0,0
- 行内样式 → 1,0,0,0
- !important：不参与权重比较，但具有最高优先级（除非被更高的 !important 覆盖）

如何调整权重：

- 提高选择器的精确度（如加 class 或 id）
- 使用 !important（不推荐大量使用）

Q3：在给定宽高情况下如何实现盒子的水平垂直居中？

```
.parent {  
  position: relative;  
}  
.child {  
  position: absolute;  
  top: 50%;  
  left: 50%;  
  transform: translate(-50%, -50%);  
}
```

或使用 Flex 布局：

```
.parent {  
  display: flex;  
  justify-content: center;  
  align-items: center;  
}
```

Q4：不给定宽高的情况下如何居中？

推荐 Flex 布局：

```
.parent {  
  display: flex;  
  justify-content: center;  
  align-items: center;  
}
```

可根据内容自适应宽高，不需要固定尺寸。

Q5：JavaScript 有哪些数据类型？

基本类型（值类型）：

- `undefined`
- `null`
- `boolean`
- `number`
- `string`
- `bigint`
- `symbol`

引用类型：

- `object`（包括：Array、Function、Date、RegExp 等）

Q6：如何判断 JS 中数据类型？

常见方式：

1. `typeof`：用于判断基本类型，不能区分 `null` 和对象。
2. `Object.prototype.toString.call(val)`：
 - 精确判断所有类型，例如：

```
Object.prototype.toString.call([]); // "[object Array]"
```

3. `Array.isArray(val)`：判断是否数组。
4. `val instanceof Constructor`：判断对象是否为某个构造函数的实例（受原型链影响）。

Q7：原型与原型链理解

- 每个对象都有一个 `__proto__` 指向其构造函数的 `prototype` 属性。
- 构造函数的 `prototype` 中定义的属性和方法会被其实例继承。
- 当访问一个对象的属性或方法时，会沿着原型链向上查找，直到找到或到 `null` 结束。

例如：

```
function Person() {}  
let p = new Person();  
p.__proto__ === Person.prototype; // true
```

Q8：数组常见方法，是否改变原数组？

方法	是否改变原数组	说明
<code>push</code>	☑	尾部添加元素
<code>pop</code>	☑	删除最后一个元素
<code>shift</code>	☑	删除第一个元素
<code>unshift</code>	☑	首部添加元素
<code>splice</code>	☑	添加/删除任意位置的元素
<code>sort</code>	☑	排序，会改变顺序
<code>reverse</code>	☑	反转数组
<code>concat</code>	✗	合并数组，返回新数组
<code>slice</code>	✗	截取数组片段，返回新数组
<code>map</code>	✗	映射新数组
<code>filter</code>	✗	过滤新数组
<code>reduce</code>	✗	累加、聚合，返回新值
<code>forEach</code>	✗	遍历无返回，不影响原数组内容

Q9：防抖与节流理解

防抖 (Debounce)：

- 高频触发只执行最后一次
- 常用于输入框联想、按钮提交

```
function debounce(fn, delay) {
  let timer;
  return function (...args) {
    clearTimeout(timer);
    timer = setTimeout(() => fn.apply(this, args), delay);
  };
}
```

节流 (Throttle)：

- 固定时间内只执行一次
- 常用于滚动监听、resize 等事件

```
function throttle(fn, limit) {
  let lastTime = 0;
  return function (...args) {
    const now = Date.now();
    if (now - lastTime >= limit) {
      lastTime = now;
      fn.apply(this, args);
    }
  };
}
```

Q10：浏览器存储方式及区别

存储方式	大小限制	生命周期	特点
Cookie	~4KB	手动/过期时间	每次请求自动携带到服务端
LocalStorage	~5MB	永久（除非手动清除）	不随请求发送，持久存储
SessionStorage	~5MB	会话级（关闭页面失效）	页面刷新可用，关闭浏览器失效
IndexedDB	数百MB	持久	结构化数据存储，异步操作

京东2

Q1：Promise 的特性

1. 三个状态：pending → fulfilled / rejected，且状态一旦改变不能再变；
2. 异步执行：then、catch、finally 总是异步回调（微任务）；
3. 链式调用：then 可返回新 Promise，实现链式调用；
4. 错误传递机制：支持异常捕获和中断（catch 或 try/catch）；
5. 不可取消（本身）；

Q2：异步队列和后端并发优化

📌 “异步队列解决后端并发受限”是指：

后端瞬时并发量很大，服务压力大 → 前端通过任务排队（队列）异步执行，比如上传、请求等排队等待服务资源，避免请求雪崩。

🔑 拓展场景：你说的“CPU 能跑 10 个任务”，优化方式是：

- 浏览器端/Node端控制并发数：使用 Promise.allSettled + 限流控制 方案；
- 使用任务池/信号量实现并发控制（如 p-limit, async-pool）；
- web worker 多线程计算 offload 到 CPU 多核；

Q3：HTTPS 加密原理 & CA 证书流程

1. HTTPS 使用 TLS 协议；
2. 使用的是混合加密：
 - 非对称加密：客户端获取服务端公钥；
 - 对称加密：客户端用公钥加密随机生成的对称密钥（如 AES）；

- 后续通信使用对称密钥，性能高；

🔑 CA 证书作用：

- 服务器将公钥与身份信息打包生成证书，交给权威 CA 签名；
- 浏览器内置 CA 机构公钥，校验服务端证书是否可信；

🔑 浏览器怎么获取 CA 公钥：

- 浏览器厂商在发布版本时内置了受信 CA 证书列表；
- 比如 Chrome 浏览器的可信根证书列表是写在源码里的。

Q4: watchEffect 的实现原理

- `watchEffect(fn)` 会在调用时自动执行一次；
- 收集在 `fn` 中访问的响应式数据作为依赖；
- 数据变动时，重新执行 `fn()`，这就是响应式追踪机制。

`fn` 是你定义的一个函数，它会在执行时自动捕获函数中使用的响应式数据。当这些数据发生变化时，`watchEffect` 会重新执行 `fn`，实现响应式效果。

🌟 本质：

```
effect(() => {  
  fn(); // 执行并追踪其中的依赖  
});
```

- Vue3 利用 `effect` + `proxy` 的 `getter` 拦截 → `track()` → 收集依赖；
- 修改数据时 → `trigger()` → 重新执行依赖的 `effect`。

Q6: Vue 动态路由鉴权 + 用户信息存储在哪？

- 鉴权方式：在 `router.beforeEach` 中判断用户是否登录，是否有权限访问当前路由；
- 用户信息存储位置：
 - 登录后拿到 token（放 `localStorage` / `sessionStorage`）；
 - 用户基本信息（可存 `store` 或 `sessionStorage`）；
 - 建议 token 放 `localStorage` + 请求时添加到请求头；

✅ 安全建议：

- 前端加密不可防止被盗用；
- token 存储仍需防止 XSS → 加强 CSP + 输入过滤；
- 重要信息不要长期存在本地，短期授权 + 滑动过期更稳妥；

Q7: Webpack 无法加载 png 图该怎么办？

如果 Webpack 无法加载 `.png` 图片，通常是由于缺少正确的配置或者相关的加载器。你可以按照以下步骤进行排查和配置：

1. 安装必要的依赖

Webpack 默认并不支持加载图片文件，需要使用加载器（loader）来处理。你需要安装一些依赖，例如 `file-loader` 或 `url-loader` 来处理 `.png` 文件的加载。

```
npm install --save-dev file-loader url-loader
```

或者，Webpack 5+ 中建议使用 `asset modules`，不需要额外安装 `file-loader` 或 `url-loader`。

2. 配置 Webpack

你可以通过修改 `webpack.config.js` 文件来配置如何加载 `.png` 文件。

方案 1：使用 `file-loader` 或 `url-loader` (Webpack 4及以下)

在 Webpack 配置中加入 `file-loader` 或 `url-loader` 配置：

```
module.exports = {
  module: {
    rules: [
      {
        test: /\.png$/, // 匹配 png 文件
        use: [
          {
            loader: 'file-loader', // 使用 file-loader 处理图片
            options: {
              name: '[name].[hash].[ext]', // 设置图片名称，支持哈希值
              outputPath: 'images/', // 设置输出目录
            }
          }
        ]
      }
    ]
  }
};
```

方案 2：使用 Webpack 5 的 `asset modules` (推荐)

Webpack 5 引入了新的资源模块（Asset Modules），使得图片处理更加简化。你可以使用 `type: 'asset/resource'` 来处理 `.png` 文件：

```
module.exports = {
  module: {
    rules: [
      {
        test: /\.png$/, // 匹配 png 文件
        type: 'asset/resource', // 使用资源模块处理图片
        generator: {
          filename: 'images/[name].[hash][ext][query]', // 设置图片的输出路径和名称
        }
      }
    ]
  }
};
```

方案 3：使用 `url-loader`（用于小文件）

如果你的 `.png` 文件比较小，你可以使用 `url-loader`，它会把图片文件转换为 base64 字符串嵌入到 JS 文件中。这样可以减少 HTTP 请求数，但适用于小图片。

```
module.exports = {
  module: {
    rules: [
      {
        test: /\.png$/,
        use: [
          {
            loader: 'url-loader',
            options: {
              limit: 8192, // 文件大小限制，超过该限制会使用 file-loader
              name: '[name].[hash].[ext]',
              outputPath: 'images/',
            }
          }
        ]
      }
    ]
  }
};
```

3. 检查图片路径

确保你在代码中引用图片的路径是正确的。例如：

```
import image from './path/to/image.png'; // 使用相对路径或绝对路径
```

如果使用相对路径，确保图片的实际路径和代码中的路径一致。

4. 检查输出文件

确保你的构建文件已经生成了正确的图片文件。如果配置正确，图片文件应当被输出到 `images/` 文件夹或你指定的路径下。

5. 清理缓存和重新构建

如果你在之前的构建中遇到问题，可以尝试清理缓存并重新构建：

```
npm run clean # 如果你有清理命令
npm run build
```